

DonorPerfect Online XML API Documentation

User Manual

SofterWare, Inc.

Table of Contents

1.	Executive Summary	9
	Intended Audience	9
	Version Control	9
2.	API Overview	10
	What is the DP API?	10
	Common API Uses	10
	Before You Begin.....	10
	How do I use the DP API?	11
	API Example Call Breakdown	11
	How to Obtain a DonorPerfect API Key	13
	Note re URLEncoding	13
	Desired API call:	13
	Example of coding for this:	14
	Resulting value of \$encoded_APIcall:	14
	Note re Advanced Password Security	14
	Sample API Call.....	15
	Using Named Parameters.....	16
	dp_savedonor example:	16
	dp_savegift example:.....	16
	API Administration and Limitations	17
	Row Retrieval Limit.....	17
	Logging	18
	No Concurrent User Blocking	18
	Change Notification	18
3.	Dynamic Queries.....	19
	SELECT Statements.....	19
	DP Table ListS	20

4. Predefined Procedures	22
dp_donorsearch	22
Introduction	22
Parameters:	22
Sample Call:.....	23
Returns:	23
Notes:.....	23
dp_savedonor	24
Introduction	24
Parameters:	24
Sample Call:.....	25
Returns:	25
Notes:.....	25
dp_gifts	26
Introduction	26
Parameters:	26
Sample Call:.....	26
Returns:	26
Notes:.....	27
dp_savegift	27
Introduction	27
Parameters:	27
Sample Call:.....	30
Returns:	30
Notes:.....	30
dp_savepledge	31
Introduction	31
Parameters:	31
Sample Call:.....	33
Returns:	33
Notes:.....	33
dp_savecontact	34
Introduction	34
Parameters:	34
Sample Call:.....	35

Returns:	35
Notes:.....	35
dp_saveotherinfo	35
Introduction	35
Parameters:	35
Sample Call:.....	36
Returns:	36
Notes:.....	36
dp_saveaddress	36
Introduction	36
Parameters	36
Sample Call.....	38
Returns	38
dp_save_udf_xml	38
Introduction	38
Parameters:	38
Sample Call:.....	39
Returns:	39
Notes:.....	39
dp_savecode.....	40
Introduction	40
Parameters	40
Sample Call.....	41
Returns	42
Notes.....	42
dp_savelink.....	42
Introduction	42
Parameters:	42
Sample Call:.....	43
Returns:	43
Notes:.....	43
dp_savemultivalue_xml.....	43
Introduction	43
Parameters:	44
Sample Call:.....	44

Returns:	44
Notes:.....	44
mergemultivalues.....	45
Introduction	45
Parameters:	45
Sample Call:.....	45
Returns:	45
Notes.....	45
dp_deletemultivalues_xml	46
Introduction	46
Parameters:	46
Sample Call:.....	47
Returns:	47
Notes:.....	47
dp_saveflag_xml	47
Introduction	47
Parameters:	48
Sample Call:.....	48
Returns:	48
Notes:.....	48
dp_delflags_xml	49
Introduction	49
Parameters:	49
Sample Call:.....	49
Returns:	49
Notes:.....	49
dp_tribAnon_MyTribSummary.....	50
Introduction	50
Parameters	50
Sample Call:.....	50
Returns:	50
Notes.....	50
dp_tribAnon_Search.....	51
Introduction	51
Parameters	51

Sample Call.....	51
Returns	51
Notes.....	52
dp_tribAnon_Create.....	52
Introduction	52
Parameters	52
Sample Call:.....	53
Returns:	53
Notes.....	53
dp_tribAnon_AssocTribToGift.....	54
Introduction	54
Parameters	54
Sample Call:.....	55
Returns:	55
Notes.....	55
dp_tribAnon_SaveTribRecipient	55
Introduction	55
Parameters	56
Sample Call.....	56
Notes.....	56
dp_tribNotif_Save	57
Introduction	57
Parameters	57
Sample Call.....	58
Returns	59
Notes.....	59
dp_tribAnon_Update.....	59
Introduction	59
Parameters	60
Sample Call.....	61
Returns	61
Notes.....	61
dp_PaymentMethod_Insert	61
Introduction	61
Parameters:	62

Sample Call:.....	62
Returns:	62
Notes:.....	63
dp_PaymentMethod_Update.....	64
Introduction	64
To use this procedure	64
Parameters:	65
Sample Call:.....	65
Returns:	65
Notes:.....	66
5. Supplemental Information.....	67
Gift Aid Program	67
Soft Credits	67
Split Gifts	68
Pledge Notes.....	69
Checkbox Fields.....	70
Example:	70
Sample output:.....	71
Get a list of possible field values	71
Set a value	71
Delete a value	71
Gift Adjustments	71
Created_date and modified_date fields	75
Tributes API Calls	75
Unicode Support	76
Quebec (QC) Law 25	77
How to access these via the API	77
Test for field presence:	78
6. Support and Implementation Services	80

1. Executive Summary

The DonorPerfect Online (DP) Application Programming Interface (API) makes it possible for you to integrate DonorPerfect Online with your existing web environment. You can, for example, access donor and gift information on secure (API key protected) portions of your intranet.

The API proves secure connections to the DP database, allowing read and write access in real time.

Common uses of the API allow developers to create content rich applications for web sites to display donor and gift information. For example, a religious organization may choose to display a list of church contact records from the DonorPerfect Online database on their website. Then, when the contact records are updated in the DonorPerfect Online database, the website will automatically display the updated records. Extending the same scenario, the API could also be configured to allow users to update the contact records from their website.

If you are not sure whether your DonorPerfect configuration includes the API, contact your DonorPerfect account representative!

Intended Audience

This document is written for customers who currently have an existing DonorPerfect Online license, or for those developers that wish to integrate their third party applications with the overall DP CRM solution.

Version Control

This document has been updated to Version 7.1 to correspond to DonorPerfect Online Version 2024.02.16. This update includes the following changes:

- Addition of @currency field to dp_savepledge API call
- Removal of one option from mergemultivalues API call
- Addition of note to @receipt field in dp_savegift

2. API Overview

What is the DP API?

The DonorPerfect Online (DP) Application Programming Interface (API) is a set of calls and parameters that one can use to interface external applications with DP. The API uses XML to return data back to the client, and these XML data 'streams' can be easily parsed using many popular XML parsers. Simply put, the DP API is a way for programmers and webmasters to make their existing websites or any other applications synchronize with DP on a variety of levels.

Common API Uses

The DP API includes functions such as:

- Saving Donor/Constituent Information – includes demographic Name, Address, Email, Phone Numbers, etc.
- Saving a Donor/Constituent's extended information (User Defined Fields) – Unlimited ability to save to any field in the DP system at the donor/constituent level- including ones that are created by the client.
- Retrieving a Donor/Constituent's record from the database – retrieves all demographic information and User Defined Fields.
- Searching for a Donor/Constituent, and returning the Donor_ID.
- Saving a Gift – saves all aspects of a gift or pledge transaction, including Date, Start Date, Amount, Gift Type, General Ledger, Solicitation, Thank You Letter Type, Memo/Comments, etc.
- Saving a Gift's extended information (User Defined Fields) – unlimited ability to save any field at the Gift/Pledge level.
- Listing Gifts for a selected Donor/Constituent – retrieves Gift or Pledge Transactions for the selected Donor.

The list of these functions is constantly expanding as clients request more commonly used functions exposed to the API. It may also possible to have a custom call designed for your organization, but that would have to be arranged through your Account Manager.

Before You Begin

It is important to understand that the DonorPerfect Online (DP) Application Programming Interface (API) comprises a set of API calls that can be used within application programming code written by, or for, your organization. Your organization is responsible for the development, testing, implementation, and maintenance of your API based application(s) and you should be aware that there are certain limitations involved with using the API:

- There is a row retrieval which is described in the API Administration and Limitations section of this document.
- Predefined Procedures should be used where possible rather than Dynamic API calls
- Limited SQL functionality is supported in Dynamic API calls and no SQL functionality beyond that which is represented in this API User Manual is supported.
- SmartActions are not supported via the API – i.e.; changes to the DonorPerfect data made via the API will not trigger SmartActions.
- Acknowledgement response time for support calls is not guaranteed and is independent of any other support guidelines. Calls are answered in the order they are received and under normal circumstances, it's reasonable to expect a response within one business day. For emergencies, send an additional email to support@donorperfect.com. DonorPerfect support services extend to your use of API calls but do not include supporting your API based application.
- SofterWare makes every effort to maintain the constant availability of the API connection but is unable to guarantee a specific service level guarantee due to factors such as network congestion and transient API traffic levels.
- While we avoid doing so when we can, SofterWare reserves the right to change API interface as needed to service the greater needs of our clients.

How do I use the DP API?

The DP API can be accessed via HTTPS (Hyper Text Transfer Protocol Secure) requests made to a specific page located on the secure DP web server. Anyone who has familiarity with XML and has possibly used an outside credit card processor (which requires an HTTPS request) should be comfortable with using the DP API. XML is a structured data set that is sent from our DP system to your application (website, etc.) that is then translated to a meaningful form whether for the web user to view or to be stored by your application (database, file, email, etc.). Some calls are used exclusively to save information in DP such as Main, Gift, Pledge or Contact screen information.

API Example Call Breakdown

A sample HTTPS request is analyzed below. This one returns all the gifts that are in the DP system for a donor with id=1269525. This example assumes a API Key of 'xxxxx':

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxx&action=dp_g  
ifts&params=@donor_id=640
```

Each XML API call has a number of key components:

1. **The Preamble:** <https://www.donorperfect.net/prod/xmlrequest.asp> - a constant location of the page that accepts the parameters and returns the requested XML information.
2. **The Action Statement:** **action=dp_gifts** – action (function) that one would like to perform. The Action Statement may either be one of the predefined procedures or a SELECT statement
3. **The Parameters:** **¶ms=@donor_id=640** – a list of comma separated parameters. This particular example only has one parameter but in most cases there will be more than one for other functions (especially that deal with saving information into DP). These may include numeric, date and text parameters. Enclose any date or text parameters in single quotes as shown.
4. **Connection credentials:** These are in the form of a DonorPerfect API key that is created and provided by DonorPerfect Support (or the API Desk)
 - Each API call includes API connection credentials in the form of an API key
 - The API key is entered as: `apikey=xyz123....`
 - The API key should not be placed in quotes
 - **The API key must not be URLEncoded**
 - The API key can either follow the Preamble or at the end of the call (after action statement and parameters)
 - Every API Key is associated with a DonorPerfect User account
 - i. The important point to understand about this is that while your connection is associated with a DonorPerfect user account BUT your API calls have **no** association whatsoever with user permissions so the limitations of individual users have no bearing on what API calls could be run from an account with limited permissions

There are two ways to input parameters:

The old way, and still supported, is to use **unnamed parameters** listed in order and separated by a comma (,).

```
...&params=1, 'Apple', '01/15/2018
```

This method works but all fields must be in correct order and is more difficult and time consuming for you to debug if your API call doesn't work properly the first time.

The NEW and RECOMMENDED way is using Named Parameters:

... ¶ms=@this=1, @that='Apple', @somedate='01/15/2018'

- When you use named parameters, the order in which they are listed is flexible
- Using named parameters does **not** allow you to skip required parameters
- You cannot mix named parameters and unnamed parameters within a single API call
- Additional information on using named parameters in the Using Named Parameters section of this document.
- Date values are always single quoted and in the mm/dd/yyyy format
- Non numeric parameters must be enclosed in single quotes

How to Obtain a DonorPerfect API Key

To get an API Key value for your DonorPerfect site, request one through your regular DonorPerfect support channel or contact the API Help Desk (api@softerware.com).

If you are an API integrator acting on behalf of a DonorPerfect Online (DP) client, please have your DP client request the API Key.

Note re URLEncoding

When your application sends an API call to DonorPerfect, the parameters (or SQL statement e.g.; if you are running a SELECT query) of the API call should be URLEncoded:

Here's an example of PHP statements URLEncoding an API call:

SAMPLE QUERY FOR URLENCODE

Desired API call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxxxxx
xxxxxxxx&action=dp_save_udf_xml&params=@matching_id=640,
@field_name='MRWMEMO',@data_type='C', @char_value='Basic math
says 2 + 3 = 5', @date_value=null, @number_value=null,
@user_id='APITestCall'
```

Example of coding for this:

```
$preamble = "https://www.donorperfect.net/prod/xmlrequest.asp?apikey=";  
$key = "xxxxxxxxxxxxxxxxxxxxxx";  
$action = "&action=dp_save_udf_xml&params=";  
  
$payload="@matching_id=640, @field_name='MRWMEMO',@char_value='Typing in  
some characters that will be urlencoded like this = sign, this + sign  
and a $ sign', @date_value=null, @number_value=null,  
@user_id='APITestCall'";  
  
$encoded_APIcall = $preamble.$key.$action.urlencode($payload);  
  
-----
```

Resulting value of \$encoded_APIcall:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxxxxxxxxxxxxxx  
xxxxx&action=dp_save_udf_xml&params=%40matching_id%3D640%2C%20%40field_n  
ame%3D%27MRWMEMO%27%2C%40data_type%3D%27C%27%2C%20%40char_value%3D%27Bas  
ic%20math%20says%20%20%2B%20%3D%20%5%27%2C%20%40date_value%3Dnull%2C  
%20%40number_value%3Dnull%2C%20%40user_id%3D%27APITestCall%27
```

Please note: The above example uses the PHP URLEncode function.

Note re Advanced Password Security

Under DonorPerfect Settings (cog icon) > Parameters > General Parameters, there are options for enhanced password security:

Enable advanced password security ☐
Password Frequency (days)
Password Expiration Notification (days)

User password expiry does **NOT** have any effect on the associated API key (if configured).

Sample API Call

A sample HTTPS request is analyzed below. This one returns all the gifts that are in the DP system for a donor with id=1269525. The actual key values are much longer than the partial key shown here but does not exceed 200 characters:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxx.....
&action=dp_gifts&params=@donor_id=1269525
```

Example of results when entered in address bar of a browser:

```
- <result>
- <record>
  <field name="gift_date2" id="gift_date2" value="09/30/2005" />
  <field name="amount" id="amount" value="5000" />
  <field name="total" id="total" value="0" />
  <field name="sub_solicit_code" id="sub_solicit_code" value="" />
  <field name="campaign" id="campaign" value="KA0905" />
  <field name="balance" id="balance" value="0" />
  <field name="gl" id="gl" value="" />
  <field name="solicit_code" id="solicit_code" value="0223AF" />
  <field name="reference" id="reference" value="56724" />
  <field name="record_type" id="record_type" value="G" />
  <field name="gift_id" id="gift_id" value="384441" />
  <field name="donor_id" id="donor_id" value="1269525" />
</record>
- <record>
  <field name="gift_date2" id="gift_date2" value="09/28/2005" />
  <field name="amount" id="amount" value="5000" />
  <field name="total" id="total" value="0" />
  <field name="sub_solicit_code" id="sub_solicit_code" value="" />
  <field name="campaign" id="campaign" value="KA0905" />
  <field name="balance" id="balance" value="0" />
  <field name="gl" id="gl" value="" />
  <field name="solicit_code" id="solicit_code" value="0223AF" />
  <field name="reference" id="reference" value="56724" />
  <field name="record_type" id="record_type" value="G" />
  <field name="gift_id" id="gift_id" value="384382" />
  <field name="donor_id" id="donor_id" value="1269525" />
</record>
</result>
```

Using Named Parameters

Some of the examples in this document specify parameter values based on their position. For example, the example shown for the `dp_savedonor` API call specifies parameters in this way. The parameter names are not specified but are listed in order and are separated by commas. The drawback with this method is that it is more difficult and time consuming to debug these API calls if they don't work properly the first time:

```
https://www.donorperfect.net/prod/xmlrequest.asp?action=dp_savedonor&params=0,'Orianthi','Panagaris', null, null, null, null, null, null, '4240 Main St.', null, 'North Woodstock', 'NH', '12345', 'US', null, '205-555-1212', null, null, '205-987-6543', 'orianthi@bigstar.com', null, null, 'N', null, null, 'API User' &apikey=xxxxxx
```

The preferred method, however, is to enter parameters using the parameter names instead of entering them by position. When you use named parameters in an API call, you do not need to follow a specific order as long as you enter all the fields that DonorPerfect needs to process the command. You will find that this method makes it easier for you to debug your API calls.

dp_savedonor example:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxxxxxxxxxxxxxxx&action=dp_savedonor&params=@donor_id=0 ,@first_name='Orianthi' ,@last_name='Panganaris' ,@middle_name=null ,@suffix=null ,@title=null ,@salutation=null ,@prof_title=null ,@opt_line=null ,@address='4240 Main St.' ,@address2=null ,@city='North Woodstock' ,@state='NH' ,@zip='12345' ,@country='US' ,@address_type=null ,@home_phone='205-555-1212' ,@business_phone=null ,@fax_phone=null ,@mobile_phone='205-987-6543' ,@email='orianthi@bigstar.com' ,@org_rec='N' ,@donor_type='IN' ,@nomail='N' ,@nomail_reason=null ,@narrative='Narrative field contents here' ,@user_id='My AppName'
```

dp_savegift example:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxxxxxxxxxxxxxxx&action=dp_savegift&paams=@gift_id=0,@donor_id=16, @record_type='G', @gift_date='2017/05/02',@amount=23.45,@gl_code='GEN', @solicit_code='BQ10', @sub_solicit_code='TS', @gift_type='VISAIC', @split_gift='N',@pledge_payment='N',@reference=null, @transaction_id='1234567890123',@memory_honor=null, @gfname=null,@glname=null,@fmv=0,@batch_no=0,@gift_narrative='Gift narrative field value.' ,@ty_letter_no='TY' ,@glink=null,@plink=null,@nocalc='N' ,@receipt='N' ,@old_amount=null ,@user_id='My AppName'
```


API Administration and Limitations

Row Retrieval Limit

There is a 500 row retrieval limit for data retrieved via API calls from DonorPerfect.

To avoid encountering this limit, modify your queries to pull subsets of the desired data where applicable.

Example1: Retrieval of DPCODES Solicitation Codes

```
...&action=select top 500 code, description from dpcodes where  
field_name='SOLICIT_CODE' order by code  
  
...&action=select top 500 code, description from dpcodes where  
field_name='SOLICIT_CODE' and code > '{last code}' order by code  
  
etc... (i.e.; repeats of 2nd API call until all values have been retrieved)
```

- where {last code} is last CODE value retrieved in the previous query

Example 2: Retrieval of Recent Gift Fields

```
...&action=select top 500 gift_id, amount, created_date, modified_date,
gl_code from dpgift WHERE COALESCE(DPGIFT.MODIFIED_DATE,
DPGIFT.CREATED_DATE) >='{date}' order by gift_id

...&action=select top 500 gift_id, amount, created_date, modified_date,
gl_code from dpgift WHERE COALESCE(DPGIFT.MODIFIED_DATE,
DPGIFT.CREATED_DATE) >='{date}' and gift_id > {last gift_id} order by
gift_id

etc...
```

- where {date} is the last time you ran this - e.g.: yesterday's date and {last gift_id} is the highest gift_id number retrieved in the previous query
- the COALESCE function is an SQL statement returns the first value that does not evaluate to NULL. So if this is a new item with no modified_date, it returns the created_date. If, on the other hand, it is an item that has been updated, there will be a modified_date value and it will return that.

Logging

DonorPerfect Online logs all API requests for record keeping and administration purposes. These logs are kept to help troubleshoot common problems as well as to identify fraudulent or suspicious activity.

No Concurrent User Blocking

The API is designed such that the API user is exempted from the DP restriction on the maximum number of concurrent users, so API calls are never blocked regardless of the number of concurrent users connected to the DP system.

Change Notification

Any changes to the DonorPerfect API will be noted in the DonorPerfect Community Release Notes:

<https://software.force.com/dpcommunity/s/global-search/%22Release%20Notes%22>

Additional details on these changes, as appropriate, will be documented in this API User Manual. The Version Control section of this document lists any changes since the previous release of the document.

3. Dynamic Queries

SELECT Statements

This query format is based on the ANSI¹ SQL implementation of Structured Query Language (SQL). SQL is the universal language used to communicate with databases.

The dynamic query feature of the DP API allows users to build API calls using SQL syntax.

DonorPerfect runs on Microsoft SQL Server 2019, so your SQL syntax should be compatible with their syntax rules.

Example:

DP contains a table called DP which contains donor information including people's first and last names and their donor ID numbers. The column names are DONOR_ID, FIRST_NAME and LAST_NAME. The values are capitalized for clarity, but the query does not expect capitalization.

An SQL query to retrieve DONOR_ID, FIRST_NAME and LAST_NAME from the DP table for people with the last name 'Bacon' would look like this:

```
Select donor_id, first_name, last_name from dp where last_name = 'bacon'
```

Which, by the way, is identical to sending this API call:

```
SELECT DONOR_ID, FIRST_NAME, LAST_NAME FROM DP WHERE LAST_NAME = 'BACON'
```

This SQL gets placed into an XML API call and follows the `&action =` parameter like this:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxx&action=
Select donor_id, first_name, last_name from dp where last_name = 'bacon'
```

Please note that you can include multiple tables in queries including the use of JOIN statements. For example:

```
Select d.donor_id, d.first_name, d.last_name, du.mcat from dp d, dpudf du
where d.donor_id = du.donor_id and last_name = 'bacon'
```

¹ ANSI is the American National Standards Institute. MSSQL uses ANSI SQL.

If there is a donor in your system named Kevin Bacon and the above command was submitted through a web browser with the appropriate username and password, the raw (unprocessed) XML result will look like this:

```
- <result>
- <record>
  <field name="donor_id" id="donor_id" value="68" />
  <field name="first_name" id="first_name"
    value="Kevin" />
  <field name="last_name" id="last_name"
    value="Bacon" />
</record>
</result>
```

To retrieve all fields from the DP table, you would use the asterisk wildcard (*) instead of field names.

NOTE: This can be a good way to identify all the field names in a table **BUT** make sure you limit the result set by specifying a particular donor_id or gift_id as appropriate. Otherwise your system will be burdened with a query retrieving thousands of data values. The query to retrieve all DP table field data for Kevin (donor_id 68) would look like this:

```
https://www.donorperfect.net/prod/xmlrequest.asp?action= Select * from dp
where donor_id = 68 &apikey=xxxxxx
```

While retrieving all fields for a particular donor or gift with a SELECT * query can be useful in a test command to view all fields for a specified table, **it is not good practice to retrieve all fields in your finished application.** Doing so would slow down your application retrieval. Instead, just retrieve the fields you actually need by listing them by name:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxx&action=SELECT
DONOR_ID, FIRST_NAME, LAST_NAME, EMAIL, ADDRESS, CITY, STATE, ZIP
FROM DP WHERE LAST_NAME='Dotsaur' AND FIRST_NAME LIKE 'J%25'
```

- Note: The %25 is a multicharacter wildcard which is the URL encoded equivalent to the percent (%) sign

DP Table Lists

This is a list of tables in DP you can run dynamic queries on

Table Name	Comments
DP	Contains donor information
DPUDF	User Defined Fields associated with the DP table
DPCODES	This table contains code values shown in drop-down lists – e.g.; General Ledger code values, Solicitation codes, Campaign codes, etc.

DPUSERMULTIVALUES	Checkbox field values
DPFLAGS	Contains any <u>set</u> flag values
DPGIFT	Contains gift information
DPGIFTUDF	User Defined Fields associated with the DPGIFT table
DPADDRESS	Allows storages of additional addresses for donors (in DP Address tab)
DPADDRESSUDF	User Defined Fields associated with the DPADDRESS table
DPLINK	Associated with values seen in the DP Link tab
DPCONTACT	Holds standard fields associated with the DP Contacts tab
DPCONTACTUDF	Holds any UDFs associated with the DP contacts tab
DPTHERINFO	Associated with values seen in the DP Other Info tab
DPTHERINFOUDF	User Defined fields added to the DP Other Info tab
DPTRIBUTESANON	This table contains the list of tributes including tribute name, active status, type, and notification person name
DPTRIBUTESANON_GIFTASSOC	This table creates the link between the gift and the tribute by containing the GIFT_ID and the TRIBUTE_ID.
DPPAYMENTMETHOD	Contains payment information associated with EFT Transactions for systems where this feature has been enabled. Two examples of the data returned from this table are shown in the section on the <code>dp_paymentMethodInsert</code> command.

4. Predefined Procedures

You may notice that many of these API calls have a USER_ID field:

- **We recommend** that you use a name here, such as the name of your API application, for auditing purposes.
- The @user_id value does not need to match the name of an actual DP user account.
- The @user_id field can contain up to 20 characters
- This can be very useful if you are trying to track whether a particular entry in DP was created by your application or by another user.

dp_donorsearch

Introduction

Searching for a Donor—used to search for the donor based on several search criteria (similar to the search functionality offered in DP). You can use “%” for wildcards.

Parameters:

Parameter	Type	Notes
@donor_id	numeric	Enter as NULL to search all donors
@last_name	Nvarchar(75)	If you aren't searching for a specific field value or the field is empty, use NULL instead of ''.
@first_name	Nvarchar(50)	As above
@opt_line	Nvarchar(100)	As above
@address	Nvarchar(100)	As above
@city	Nvarchar(50)	As above
@state	Nvarchar(30)	As above
@zip	Nvarchar(20)	As above
@country	Nvarchar(30)	As above
@filter_id	numeric	Always enter this parameter as NULL
@user_id	Nvarchar(20)	If you aren't searching for a specific field value or the field is empty, use NULL instead of ''.

Sample Call:

Example 1:

https://www.donorperfect.net/prod/xmlrequest.asp?action=dp_donorsearch¶ms=null,'Pa%','Ori%','null,null,null,null,null,null,null

Example 2:

https://www.donorperfect.net/prod/xmlrequest.asp?action=dp_donorsearch¶ms=@donor_id=null,@last_name='Panagaris',@first_name='O%',@opt_line=null,@address='4240 PRS Ave.',@city='North Woodstock',@state='NH',@zip='12345',@country=null,@filter_id=null,@user_id=null

Returns:

```
<result>
-<record>
<field name="donor_id" id="donor_id" value="147"/>
<field name="first_name" id="first_name" value="Orianthi"/>
<field name="last_name" id="last_name" value="Panagaris"/>
<field name="title" id="title" value=""/>
<field name="suffix" id="suffix" value=""/>
<field name="address" id="address" value="4240 Main St."/>
<field name="opt_line" id="opt_line" value=""/>
<field name="city" id="city" value="North Woodstock"/>
<field name="state" id="state" value="NH"/>
<field name="zip" id="zip" value="12345"/>
<field name="gifts" id="gifts" value="0"/>
<field name="gift_total" id="gift_total" value="0"/>
<field name="address2" id="address2" value=""/>
<field name="donor_name" id="donor_name" value="Orianthi Panagaris"/>
<field name="city_state_zip" id="city_state_zip" value="North Woodstock,
NH 12345"/>
</record>
</result>
```

Notes:

- If you require different fields to be returned, or if you need the city_state_zip or donor_name fields to appear as separate fields then use a Dynamic Query SELECT statement instead to achieve the desired result set.
- Alphanumeric fields – e.g.; Address, Zip, City – must be wrapped in single quotes as shown in the examples above
- When/if a field is empty, it is set to NULL rather than " which is why a search on " returns no values.
- Searching for NULL is the same as searching for '%'

dp_savedonor

Introduction

Saving a New/Existing Donor—used to save changes to the existing donor/constituent or save the new donor/constituent into the DP system.

Parameters:

Parameter	Type	Notes
@donor_id	numeric()	Enter 0 (zero) to create a new donor/constituent record or an existing donor_id. Please note: If you are updating an existing donor, all existing values for the fields specified below will be overwritten by the values you send with this API call.
@first_name	varchar(50)	
@last_name	varchar(75)	
@middle_name	varchar(50)	
@suffix	varchar(50)	
@title	varchar(50)	Set to NULL if no title.
@salutation	varchar(130)	
@prof_title	varchar(100)	
@opt_line	varchar(100)	Enter as NULL if other field value not required.
@address	varchar(100)	
@address2	varchar(100)	
@city	varchar(50)	
@state	varchar(30)	
@zip	varchar(20)	
@country	varchar(30)	
@address_type	varchar(30)	
@home_phone	varchar(40)	
@business_phone	varchar(40)	
@fax_phone	varchar(40)	
@mobile_phone	varchar(40)	
@email	varchar(75)	

@org_rec	varchar(1)	Enter 'Y' to check the Org Rec field (indicating an organizational record) or 'N' to leave it unchecked indicating an individual record.
@donor_type	varchar(30)	Set to 'IN' for Individual donor or 'CO' for Corporate donor. You can also check your DP system for additional DONOR_TYPE field value choices.
@nomail	varchar(1)	Must be set to 'N' or 'Y'.
@nomail_reason	varchar(30)	
@narrative	text(2147483647)	
@donor_rcpt_type	Nvarchar(1)	'I' for individual or 'C' for consolidated receipting preference
@user_id	Nvarchar(20)	We recommend that you use a name here, such as the name of your API application, for auditing purposes. The user_id value does not need to match the name of an actual DP user account.

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxx&action=dp_s
avedonor&params=@donor_id=0, @first_name='Orianthi',
@last_name='Panagaris', @middle_name=null, @suffix=null, @title=null,
@salutation=null, @prof_title=null, @opt_line=null, @address='37 Elmsley
St.', @address2=null, @city='North Woodstock', @state='NH', @zip='12345',
@country='US', @address_type=null, @home_phone='205-555-1212',
@business_phone=null, @fax_phone=null, @mobile_phone='205-987-6543',
@email='cathy@consolidate.com', @org_rec='N', @donor_type='IN',
@nomail='N', @nomail_reason=null, @narrative='Narrative field contents
here', @donor_rcpt_type='C', @user_id='MyAppName'
```

Returns:

This command returns the donor ID value of the new donor. In the case of an update, the returned donor_id value is always zero (0).

```
<result>
  <record>
    <field name="" id="" value="147" />
  </record>
</result>
```

Notes:

- 147 is the donor_id of the created donor
- Do not have any spaces in the first part of your command (https://www.donorperfect.net/prod/xmlrequest.asp?action=dp_savedonor¶

ms=) otherwise the API will return an error indicating that the parameter @donor_id was not supplied.

- The null values act as placeholders and must be included in your command
- Character values and date values must be enclosed in single quotes (e.g; '11/01/2010')
- The @user_id value is just there for transaction reference purposes and is not validated against the list of DP users.
- If you need to save an apostrophe in a name – e.g.; O'Reilly – you must send two apostrophes in a row – e.g; O'' Reilly . This tells the API handler that to interpret this as an apostrophe within the name rather than as the end of a quoted text value.

dp_gifts

Introduction

This procedure returns a predefined set of fields associated with all gifts given by the specified donor.

Parameters:

Parameter	Type	Notes
@donor_id	numeric	

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?action=dp_gifts&params=@donor_id=147&apikey=xxxxx
```

Returns:

```
<result>
-<record>
<field name="gift_date2" id="gift_date2" value="10/27/2010"/>
<field name="amount" id="amount" value="14.98"/>
<field name="total" id="total" value="0"/>
<field name="sub_solicit_code" id="sub_solicit_code" value="TS"/>
<field name="campaign" id="campaign" value=""/>
<field name="balance" id="balance" value="0"/>
<field name="gl" id="gl" value="NO Rcpt: BOOKS & TAPES"/>
<field name="solicit_code" id="solicit_code" value="BQ10"/>
<field name="reference" id="reference" value=""/>
<field name="record_type" id="record_type" value="G"/>
```

```
<field name="gift_id" id="gift_id" value="10230"/>
<field name="donor_id" id="donor_id" value="147"/>
<field name="anongift" id="anongift" value=""/>
<field name="gift_aid_date" id="gift_aid_date" value=""/>
</record>
</result>
```

Notes:

- If you need different or additional fields returned from the system, use a Dynamic Query SELECT command.

dp_savegift

Introduction

This procedure is used to save changes to an existing gift or to save a new gift into the DP system

Parameters:

Parameter	Type	Notes
@gift_id	numeric()	Enter 0 in this field to create a new gift or the gift ID of an existing gift. Please note: If you are updating an existing gift, all existing values for the fields specified below will be overwritten by the values you send with this API call.
@donor_id	numeric()	Enter the donor_id of the person for whom the gift will be created
@record_type	varchar(1)	Set as 'G' for a regular gift or for an individual split gift entry within a split gift (i.e.; not the Main top level split), 'P' for Pledge, 'M' for the Main gift in a split gift
@gift_date	datetime()	Set as 'MM/DD/YYYY' only. Setting time values is <u>not</u> supported.
@amount	money()	
@gl_code	varchar(30)	If desired, enter the CODE value of the General Ledger code this gift will be associated with. Code values can be found in the DPCODES table.

Parameter	Type	Notes
		Note: If you are not setting a <code>gl_code</code> value in this field, set the field to null. Do not ever set this field to empty ('').
@solicit_code	varchar(30)	If desired, enter the CODE value of the desired solicit code Note: If you are not setting a <code>solicit_code</code> value in this field, set the field to null. Do not ever set this field to empty ('').
@sub_solicit_code	varchar(30)	If desired, enter the CODE value of the desired sub-solicit code Note: If you are not setting a <code>sub_solicit_code</code> value in this field, set the field to null. Do not ever set this field to empty ('').
@campaign	varchar(30)	If desired, enter the CODE value of the desired campaign code Note: If you are not setting a campaign code value in this field, set the field to null. Do not ever set this field to empty ('').
@gift_type	varchar(30)	
@split_gift	varchar(1)	Set to 'Y' for each of the splits within a split gift but set to 'N' for the Main gift in a split gift or for any gift that is not a split gift.
@pledge_payment	varchar(1)	Set as 'N' if this gift is not a pledge payment or 'Y' if it is. If this gift is a pledge payment, you should also have the @plink field filled in with the pledge gift_id.
@reference	Varchar(100)	Set as @reference=null if not required or other value as needed
@transaction_id	numeric	The associated SafeSave Transaction ID number is normally entered here. This field supersedes the @reference field which was previously used for this purpose.
@memory_honor	varchar(30)	
@gfname	varchar(50)	
@glname	varchar(75)	
@fmv	money()	
@batch_no	numeric()	Set to 0 (zero) or other required batch number but this field will not accept NULL value

Parameter	Type	Notes
@gift_narrative	nvarchar(4000)	
@ty_letter_no	varchar(30)	Note: If you are not setting a ty_letter_no code value in this field, set the field to null. Do not ever set this field to empty ('').
@glink	numeric()	In a split gift (not the Main), set the glink value to the gift_id value of the Main gift in the split Also, if you are creating a soft credit gift (record_type='S'), use the GLINK field to identify the gift_id of the actual gift. Note: If you are not setting a gift_id value in this field, set the field to null. Do not ever set this field to empty ('') or zero.
@plink	numeric()	This field should be blank for a non-recurring gift but if you are creating a gift that is to be associated with a pledge, set the plink value of the gift to the gift_ID value of the associated pledge. Note: If you are not setting a gift_id value in this field, set the field to null. Do not ever set this field to empty ('') or zero.
@nocalc	varchar(1)	The standard value for this field is @nocalc='N'. This field must be set to 'N' for gifts to be reflected in the reports dashboard.
@receipt	varchar(1)	Set as 'Y' for yes or 'N' for no. Do not leave as blank or null.
@old_amount	money	
@user_id	Nvarchar(20)	We recommend that you use a name here, such as the name of your API application, for auditing purposes. The user_id value does not need to match the name of an actual DP user account.
@gift_aid_date	datetime()	This field relates to the UK based Gift Aid Program. See Supplemental Information > Gift Aid Program for more information.
@gift_aid_amt	Money()	This field relates to the UK based Gift Aid Program. See Supplemental Information > Gift Aid Program for more information.
@gift_aid_eligible_g	Char(1)	This field relates to the UK based Gift Aid Program. See Supplemental Information > Gift Aid Program for more information.

Parameter	Type	Notes
@currency	Nvarchar(3)	If you use the multi-currency feature, enter appropriate code value per your currency field - e.g; 'USD', 'CAD', etc.
@receipt_delivery_g	varchar(1)	This field sets receipt delivery preference for the specified gift. Supply one of the following single letter code values: - N = do not acknowledge - E = email - B = email and letter - L = letter
@acknowledgeref		Used in Canadian DonorPerfect systems to indicate official receipt acknowledgement preference code: 1AR - Acknowledge/Receipt 2AD - Acknowledge / Do Not Receipt 3DD - Do Not Acknowledge / Do Not Receipt

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxxx&action=d
p_savegift&params=@gift_id=0, @donor_id=640, @record_type='G',
@gift_date='3/29/2018', @amount=149.95, @gl_code='4540-N',
@solicit_code='BQ10', @sub_solicit_code='TS', @campaign='CAM2018',
@gift_type='VISAIC', @split_gift='N', @pledge_payment='N',
@reference=null, @transaction_id=1234567890123, @memory_honor=NULL,
@gfname=NULL, @glname=NULL, @fmv=0, @batch_no=0,
@gift_narrative='Gift narrative text', @ty_letter_no='TY', @glink=NULL,
@plink=NULL, @nocalc='N', @old_amount=NULL, @receipt='N', @user_id='My
App Name', @gift_aid_date=NULL, @gift_aid_amt=NULL,
@gift_aid_eligible_g=NULL, @currency='USD', @first_gift='N'
```

Returns:

```
<result>
  <record>
    <field name="" id="" value="10230" />
  </record>
</result>
```

Notes:

- 10230 is the gift_id of the created gift

- Do **not** include the @first_gift field in your API call. DonorPerfect will automatically set to either
 - 'Y' if this is the first gift for this donor
 - 'N' if this is NOT the first gift for this donor

List of calculated fields that are updated by dp_savegift. Other fields (e.g.; fiscal year LYx_ fields) are not updated.

- @last_contrib_amt
- @ytd
- @ly_ytd
- @gift_total
- @gifts
- @max_date
- @max_amt
- @avg_amt
- @yrs_donated

dp_savepledge

Introduction

This procedure is used to create, or save changes to, a pledge. It is not used for pledge payments. In DP, there is a parent pledge (which this command is used to create) that shows up in the DP pledges tab. Then, when pledge payments are made, they are created as gifts (record_type='G') using the dp_savegift procedure with a gift_type of 'G' like a regular gift, but add in a '**plink**' value with the gift_id of the parent pledge.

Parameters:

Parameter	Type	Notes
@gift_id	numeric()	Enter 0 in this field to create a new pledge or the gift ID of an existing pledge.
@donor_id	numeric()	Enter the donor_id of the person for whom the pledge is being created/updated
@gift_date	datetime()	Set as 'MM/DD/YYYY' only. Setting time values is <u>not</u> supported.

Parameter	Type	Notes
@start_date	datetime()	Set as 'MM/DD/YYYY' only. Setting time values is <u>not</u> supported.
@total	money()	Enter either the total amount to be pledged (the sum of all the expected payment amounts) or enter 0 (zero) if the pledge amount is to be collected ad infinitum
@bill	money()	Enter the individual monthly/quarterly/annual billing amount
@frequency	varchar (30)	Enter one of: M (monthly), Q (quarterly), S (semi-annually), A (annually)
@currency	Nvarchar(3)	Set as your default currency (e.g; USD/CAD/etc.) and value to be wrapped in single quotes: Example: @currency='CAD' If a currency value is not specified, this field will default to USD
@reminder	varchar (1)	Sets the pledge reminder flag
@gl_code	varchar(30)	Note: If you are not setting a gl_code value in this field, set the field to null. Do not ever set this field to empty ('').
@solicit_code	varchar(30)	Note: If you are not setting a solicit_code value in this field, set the field to null. Do not ever set this field to empty ('').
@initial_payment	varchar (1)	Set to 'Y' for initial payment, otherwise 'N'
@sub_solicit_code	varchar(30)	Note: If you are not setting a sub_solicit_code value in this field, set the field to null. Do not ever set this field to empty ('').
@writeoff_amount,	money()	
@writeoff_date	datetime()	
@user_id	varchar(20),	
@campaign	varchar(30)	Note: If you are not setting a campaign_code value in this field, set the field to null. Do not ever set this field to empty ('').
@membership_type	varchar(30)	Or NULL
@membership_level	varchar(30)	Or NULL
@membership_enr_date	datetime	Or NULL
@membership_exp_date	datetime	Or NULL

Parameter	Type	Notes
@membership_link_ID	numeric	Or NULL
@address_id	numeric	Or NULL
@gift_narrative	nvarchar(4000)	Or NULL
@ty_letter_no	varchar(30)	Or NULL
@vault_id	numeric()	This field must be populated from the Vault ID number returned by SafeSave for the pledge to be listed as active in the user interface.
@receipt_delivery_g	varchar(1)	'E' for email, 'B' for both email and letter, 'L' for letter, 'N' for do not acknowledge or NULL
@contact_id	numeric()	Or NULL

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxxx&action=d
p_savepledge&params=@gift_id=0, @donor_id=640,@gift_date='10/12/2018',
@start_date='10/15/2018',@total=600.00,@bill=30.00,@frequency='M',
@currency='CAD', @reminder='Y',@gl_code='4540-
N',@solicit_code='BQ10',@initial_payment='N',@sub_solicit_code='TS',
@writeoff_amount=0.00,@writeoff_date='12/29/2018',@user_id='MyApp_Name',@
campaign='REG', @membership_type=NULL, @membership_level=NULL,
@membership_enr_date=NULL, @membership_exp_date=NULL,
@membership_link_id=NULL, @address_id=NULL,@gift_narrative='Gift
narrative text', @ty_letter_no='TY',
@vault_id=123321,@receipt_delivery_g='E',@contact_id=NULL
```

Returns:

```
<result>
  <record>
    <field name="" id="" value="12902" />
  </record>
</result>
```

Notes:

- 12902 is the gift_id of the created gift

dp_savecontact

Introduction

This procedure saves fields to the DPCONTACT table. It will create a new or updated Contact record for the specified donor_id.

Parameters:

Parameter	Type	Notes
@contact_id	numeric()	Enter 0 to create a new record or the contact_id record number of an existing dpcontact record
@donor_id	numeric()	Enter the Donor ID of the donor for whom the contact record is to be created or retrieved
@activity_code	varchar(30)	CODE value for the Activity Code field. See DP Settings > Code Maintenance > Activity Code / Contact Screen. The required values will be listed in the Code column of the resulting display.
@mailing_code	varchar(30)	CODE value for Mailing Code field
@by_whom	varchar(30)	CODE value for the By Whom/Contact Screen field in DP Description value of selected code shows in the 'Assigned To' field of the contact record.
@contact_date	datetime	Contact / Entry Date field in DP
@due_date	datetime	Due Date field in DP. Set as 'MM/DD/YYYY' only. Setting time values here is <u>not</u> supported.
@due_time	varchar(20)	Time field in DP. Enter as 'hh:mm xx' where xx is either AM or PM - e.g; '02:00 PM'.
@completed_date	datetime	Completed Date field in DP. Set as 'MM/DD/YYYY' only. Setting time values is <u>not</u> supported.
@comment	varchar(4000)	Contact Notes field in DP
@document_path	varchar(200)	Type a URL/File Path field in DP
@user_id	varchar (20)	We recommend that you use a name here, such as the name of your API application, for auditing purposes. The user_id value does not need to match the name of an actual DP user account.

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?action=dp_savecontact&pa  
rams=0, 135, 'TE', 'BW02', 'ELF', '09/13/2012', '09/18/2012', '12:24:00  
PM', '09/28/2012', 'Your comment on this contact item.',  
'http://myorganization.com', 'APIUser' &apikey=xxxxxxx
```

Returns:

```
<result>  
-<record>  
<field name="" id="" value="123 />  
</record>  
</result>
```

Notes:

- 123 is the contact_id associated with the created value
- In this example, the contact_date is 09/13/2012. Typically the current date. The due_date is 09/18/2012 and the completed_date is 09/28/2012 but you would typically leave this field blank by entering null in this field to allow the DP User to whom the activity was assigned to mark the item as completed when they performed the required activity: e.g.;

```
https://www.donorperfect.net/prod/xmlrequest.asp?action=dp_savecontact&pa  
rams=0, 135, 'TE', 'BW02', 'ELF', '09/13/2012', '09/18/2012', '12:24:00  
PM', null, 'Your comment on this contact item.',  
'http://myorganization.com', 'API User' &apikey=xxxxxx
```

dp_saveotherinfo

Introduction

This procedure saves fields to the DPtherinfo table. It will create a new or updated 'Other Info' record for the specified donor_id.

Parameters:

Parameter	Type	Notes
@other_id	numeric()	Enter 0 to create a new record or the other_id record number of an existing DPtherinfo record

@donor_id	numeric()	Enter the donor_id for whom the record is to be created / updated.
@other_date	datetime()	Set as 'MM/DD/YYYY' only. Setting time values is <u>not</u> supported.
@comments	nvarchar(4000)	
@user_id	varchar(20)	We recommend that you use a name here, such as the name of your API application, for auditing purposes. The user_id value does not need to match the name of an actual DP user account.

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?action=dp_saveotherinfo&
params=0,147,'11/18/2010','my comment', 'MyAppName' &apikey=xxxxx
```

Returns:

```
<result>
-<record>
<field name="" id="" value="160" />
</record>
</result>
```

Notes:

- 160 is the other_id associated with the updated value
- This other_id value can now be used as a @matching_id value to save additional fields associated with the DPtherinfo table associated with this entry.

dp_saveaddress

Introduction

Use this API call to add or update secondary address values that appear in the DonorPerfect Online Address tab. These secondary addresses are also referred to as Seasonal Addresses in the DP Knowledge Base.

Parameters

Parameter	Type	Notes
@address_id	numeric	Enter 0 in this field to create a new address value or the address_id number of

		an existing address_id to update the existing value
@donor_id	numeric	Specify the donor_id of the donor associated with this
@opt_line	varchar(100)	Enter a secondary name or company name if appropriate
@address	varchar(100)	
@address2	varchar(100)	
@city	varchar(50)	
@state	varchar(30)	
@zip	varchar(20)	
@country	varchar(30)	
@address_type	varchar(30)	Enter the CODE value associated with the address type
@getmail	varchar(1)	Enter 'Y' or 'N' to indicate whether the Receive Mail box will be checked and to indicate whether mail can be sent to this address.
@user_id	varchar(20)	We recommend that you use a name here, such as the name of your API application, for auditing purposes. The user_id value does not need to match the name of an actual DP user account.
@title	varchar(50)	Enter a value to be stored in the Professional Title field.
@first_name	varchar(50)	
@middle_name	varchar(50)	
@last_name	varchar(75)	
@suffix	varchar(50)	
@prof_title	varchar(100)	
@salutation	varchar(130)	Enter desired salutation value (e.g.; 'Dear Bob')
@seasonal_from_date	varchar(4)	Enter the 'from' date as MMYY - e.g; November 2017 would be represented as 1117
@seasonal_to_date	varchar(4)	Enter the 'to' date as MMYY
@email	varchar(75)	
@home_phone	varchar(40)	
@business_phone	varchar(40)	
@fax_phone	varchar(40)	
@mobile_phone	varchar(40)	
@address3	varchar(100)	

@address4	varchar(100)	
@ukcounty	varchar(100)	
@org_rec	varchar(1)	Enter 'Y' to check the Org Rec field (indicating an organizational record) or 'N' to leave it unchecked to indicate an individual record.

Sample Call

```
...&action=dp_saveaddress&params='167','4220',null,'12 Main
St.',null,'Potsdam','NY','33132','USA','Home','N','UserID','title','first
_name','middle_name','last_name','suffix','prof_title','salutation','0717
','0817','email@address.com','home_phone','business_phone','fax_phone','m
obile_phone','address3','address4','ukcountry','Y'
```

Returns

This API call returns the @address_id value of the newly created (or updated) DPADDRESS value.

```
<result>
  <record>
    <field name="" id="" value="167"/>
  </record>
</result>
```

dp_save_udf_xml

Introduction

This procedure saves a Donor's extended information (User Defined Fields) —used to save changes to the user-defined fields that are custom for each client and are not part of the standard DP system.

This procedure will save a single parameter for a specified User Defined Field (UDF).

Parameters:

Parameter	Type	Notes
@matching_id	Numeric()	Specify either an existing donor_id value if updating a donor record, a gift_id value if

		updating a gift record, a contact_id number if updating a contact record or an other_id value if updating a DPtherinfo table value (see dp_saveotherinfo). If you are updating a value in the DPADDRESSUDF, specify the @address_ID as the matching ID. Also, FYI, you can link the DPADDRESS.ADDRESS_ID=DPADDRESSUDF.ADDRESS_ID in any of your SELECT queries.
@field_name	varchar(20)	
@data_type	varchar(1)	C- Character, D-Date, N- numeric
@char_value	Nvarchar(2000)	Null if not a Character field
@date_value	Datetime()	Null if not a Date field. Set as 'MM/DD/YYYY' only. Setting time values is <u>not</u> supported.
@number_value	numeric()	Null if not a Number field
@user_id	varchar(20)	We recommend that you use a name here, such as the name of your API application, for auditing purposes. The user_id value does not need to match the name of an actual DP user account.

Sample Call:

Example 1:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxxx&action=d
p_save_udf_xml&params=16013,'ALT_PHONE','C','555-
1212',null,null,'MyAppName'
```

Example 2:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxxx&action=d
p_save_udf_xml&params=@matching_id=348,@field_name='FIELDNAME',@data_type
='C',@char_value='new value here',
@date_value=null,@number_value=null,@user_id='MyAppName'
```

Returns:

```
<result>
-<record>
<field name="" id="" value=" 16013" />
</record>
</result>
```

Notes:

- 16013 is the donor_id associated with the updated value
- This example assumes that a user defined field called ALT_PHONE already exists in your system

dp_savecode

Introduction

This procedure allows you to add new code and description values to the DPCODES table. This would allow you to add things like new GL Code, Solicitation, Campaign code and other code values administered in the Code Maintenance screen of DP.

You can also use this API call to update an existing code by including the value of the existing/old code in the @original_code value and setting the @overwrite value to 'Y'.

Edit	Field Name	Code	Description	Active (Click to Change)	Delete
	EMAIL_TYPE	OTHER	Other		N/A
	EMAIL_TYPE	PERSONAL	Personal		N/A
	EMAIL_TYPE	WORK	Work		N/A

While this API call requires the specification of many fields, you will typically only supply the Field Name, Code, Description, Inactive field value and we also recommend you supply the @user_id value as well.

Parameters

Parameter	Type	Notes
@field_name	varchar(20)	Enter the name of an existing field type from the DPCODES table
@code	varchar(30)	Enter the new CODE value
@description	varchar(100)	Enter the description value that will appear in drop-down selection values
@original_code	varchar(20)	Enter NULL unless you are updating an existing code. In that case, set this field to the current (before update) value of the CODE
@code_date	Datetime()	Enter NULL
@mcat_hi	Money()	Enter NULL
@mcat_lo	Money()	Enter NULL
@mcat_gl	varchar(1)	Enter NULL
@acct_num	varchar(30)	Enter NULL
@campaign	varchar(30)	Enter NULL
@solicit_code	varchar(30)	Enter NULL

@overwrite		If you are creating a new code, set this field to NULL or 'N'. If you are updating an existing code, set this to 'Y'.
@inactive	varchar(1)	Enter 'N' for an active code or 'Y' for an inactive code. Inactive codes are <u>not</u> offered in the user interface dropdown lists.
@client_id		Enter NULL
@available_for_sol		Enter NULL
@user_id	varchar(20)	Enter the name of your API application
@cashact		Enter NULL
@membership_type		Enter NULL
@leeway_days		Enter NULL
@comments	varchar(2000)	You may enter a comment of up to 2000 characters if you like.
@begin_date	datetime	. Set as 'MM/DD/YYYY' only. Setting time values is <u>not</u> supported.
@end_date	datetime	. Set as 'MM/DD/YYYY' only. Setting time values is <u>not</u> supported.
@ty_prioritize	char(1)	Enter NULL
@ty_filter_id		Enter NULL
@ty_gift_option		Enter NULL
@ty_amount_option		Enter NULL
@ty_from_amount		Enter NULL
@ty_to_amount		Enter NULL
@ty_alternate		Enter NULL
@ty_priority		Enter NULL

Sample Call

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=...&action=dp_savecode&params=@field_name='GL_CODE', @code='4800-03',
@description='Building Fund:
Furnishings',@original_code=null,@code_date=null,@mcat_hi=null,@mcat_lo=null,
@mcat_gl=null,@reciprocal=null,@mailed=null,@printing=null,@other=null,
@goal=null,@acct_num=null,@campaign=null, @solicit_code=null,
@overwrite=null,@inactive='N',@client_id=null, @available_for_sol=null,
@user_id='My API App', @cashact=null, @membership_type=null,
@leeway_days=null,@comments=null, @begin_date=null, @end_date=null,
@ty_prioritize=null, @ty_filter_id=null, @ty_gift_option=null,
@ty_amount_option=null, @ty_from_amount=null, @ty_to_amount=null,
@ty_alternate=null, @ty_priority=null
```

Returns

```
<result>
<record>
<field name="result" id="result" value="0"/>
</record>
</result>
```

Notes

Set @inactive='N' to indicate that this entry is Active and will appear in the appropriate drop-down field in the user interface.

Notes on @original_code:

IF	THEN
@original_code = @code and the @code value already exists	All specified values will be updated for the existing code
@original_code = NULL and the @code already exists	The call returns '2' and updates nothing
@original_code = NULL or @original_code = @code but @code does not already exist	The call creates a new code

dp_savelink

Introduction

This procedure creates a link between two donors. These links show in the Links tab of the DP user interface. Note that reciprocal links are created automatically. For example, if you create a Friend link (FR) link with donor A as the donor_id value and donor B as the donor_id2 value, you will automatically see the Friend link in the DP user interface if you look at the Links tab in either donor A or donor B. Please also note that DP will create appropriate reciprocal links in the same way it does via the user interface. So, if you create a Parent link from donor A to donor B, then donor B will show donor A as a Child link.

Parameters:

Parameter	Type	Notes
-----------	------	-------

@link_id	numeric()	Enter zero (0) to create a new link entry of the link_id of an existing entry if you are updating an existing entry
@donor_id	Numeric()	Enter the donor_id of the donor where the link is being created
@donor_id2	Numeric()	Enter the donor_id of the OTHER donor involved in the link
@link_code	Varchar(30)	Enter the CODE value of the link per the Link Type values shown in the Code Maintenance screen of the DP user interface
@user_id	varchar(20)	We recommend that you use a name here, such as the name of your API application for auditing purposes. The user_id value does not need to match the name of an actual DP user account.

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?action=dp_savelink&params=0,144,135,'PA','MyApplication'&apikey=xxxxx
```

Returns:

```
<result>
<record>
<field name="" id="" value="12345"/>
</record>
</result>
```

Notes:

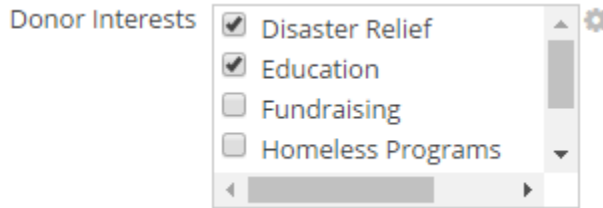
- The returned value is the link_id of the created link
- The User ID value does not have to correspond to an actual user and simply provides a reference for who checked the box. In the case of API applications, we recommend you use your API application name

dp_savemultivalue_xml

Introduction

Recommendation: Consider using the mergemultivalues API call instead of this one.

This procedure allows you to set a checkbox value. Checkbox fields in DP are stored in the DPUSERMULTIVALUES table and use a MATCHING_ID value as the index. The MATCHING_ID value corresponds to the associated DONOR_ID. Note that even though a checkbox field may contain many checkboxes, there will only be an entry present in DPUSERMULTIVALUES for checkboxes that are set (checked).



Parameters:

Parameter	Type	Notes
@matching_id	numeric	Specify the desired donor_id
@field_name	varchar(20)	Use the code value associated with the flag. For example, the 'AL', flag in this example had a description value of 'Alumni'.
@code	Varchar(30)	Enter the Code value of the checkbox entry you wish to set
@user_id	varchar(20)	We recommend that you use a name here, such as the name of your API application, for auditing purposes. The user_id value does not need to match the name of an actual DP user account.

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?action=dp_savemultivalue_xml&params=12345,'FIELDNAME','CODE','AppName'&apikey=xxxxx
```

Returns:

```
<result>
<record>
<field name="" id="" value="12345"/>
</record>
</result>
```

Notes:

- This example assumes the existence of a checkbox field with FIELD_NAME = 'FIELDNAME' and a checkbox with the CODE = 'CODE'

- The User ID value does not have to correspond to an actual user and simply provides a reference for who checked the box. In the case of API applications, we recommend you use your API application name

mergemultivalues

Introduction

This procedure allows you to set field values for an individual checkbox field. Any values not specified will be unset (unchecked).

Parameters:

Parameter	Type	Notes
@matchingid	numeric	Specify the desired donor_id
@fieldname	varchar(20)	Enter the name of the checkbox field name.
@valuestring	varchar(20)	Enter any CODE values to be set. Separate with commas. Any code values not specified will be unset (unchecked).

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxxxxxxxxxx
&action=mergemultivalues&params=@matchingid=16,@fieldname='GTYPES',@value
string='CR, RE, ST'
```

Returns:

```
<result/>
```

Notes

This is the call to use instead of `dp_deletemultivalues_xml` if you ever need to unset (uncheck) a checkbox field because it allows you to modify a single checkbox field instead of having to retrieve all checkbox fields for the screen tab and re-set them.

Depending on what you need to do, you can probably also use this instead of the `dp_savemultivalues_xml` API call as well – as it allows you to set (check) checkboxes as well as un-set them.

Also, please note that the field specifications are slightly different than the usual when you are entering the command string – e.g.; @matchingid is correct. @matching_id is incorrect.

You can retrieve a list of the possible code values for the desired field with a SELECT query like this:

```
&action=select code, description from dpcodes where field_name='{field name here}'
```

You can retrieve a list of flags set for a specific flag for a specific donor_id with a SELECT query like this:

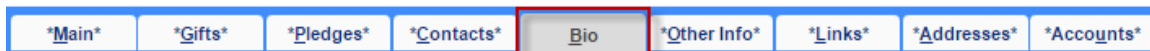
```
&action=select id, matching_id, code from dpusermultivalues where field_name='{field name}' and matching_id={donor_id}
```

dp_deletemultivalues_xml

Introduction

Recommendation: Consider using the mergemultivalues API call instead of this one.

This procedure deletes ALL checked checkbox values on the specified screen tab (e.g.; Main, Gift, Pledge, Bio, Other) for the specified donor.



Checkbox fields in DP are stored in the DPUSERMULTIVALUES table and use a MATCHING_ID value as the index. The MATCHING_ID value corresponds to the associated DONOR_ID. Note that even though a checkbox field may contain many checkboxes, there will only be an entry present in DPUSERMULTIVALUES for checkboxes that are set (checked).

We recommend that before using this API call, you use a dynamic SELECT statement to retrieve all set (checked) values for the specified donor. This will allow you to follow the delete API call with dp_savemultivalues_xml API calls to re set/check the checkbox fields you did not want deleted.

Example: ...&action=select field_name, code from dpusermultivalues where field_name='{field name}' and matching_id={donor_id}

Parameters:

Parameter	Type	Notes
-----------	------	-------

@matching_id	numeric	Specify the desired donor_id
@table_name	varchar(20)	Enter the name of the DP screen tab (i.e. MAIN, GIFT, PLEDGE, BIO, OTHER) where <u>all</u> checked fields are to be deleted
@user_id	varchar(20)	Enter a user name that will be associated with this update. Normally with will be the name you associate with your API application.

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxxxxxxxx&action=dp_deletemultivalues_xml&params=135,'BIO','Set via API'
```

Returns:

```
<result>
-<record>
<field name="" id="" value=" 135" />
</record>
</result>
```

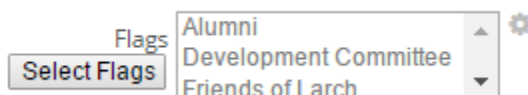
Notes:

- 135 is the donor_id associated with the updated value
- This API call removes ALL checked values from the specified DP screen tab.

dp_saveflag_xml

Introduction

This procedure allows you to set flags as shown in the top section of the Main tab. The Flags field is on the Main tab in the DP user interface.



Flags must have been previously created in Settings > Code Maintenance and the value you set corresponds to the Code value, not the description value.

Select	Edit	Field Name	Code	Description
☒		FLAG	AL	Alumni
☐		FLAG	AM	Association Member
☐		FLAG	BD	Board Member

Parameters:

Parameter	Type	Notes
@donor_id	Numeric()	Specify either a donor_id value if updating a donor record, a gift_id value if updating a gift record or an other_id value if updating a DPtherinfo table value (see dp_saveotherinfo)
@flag	varchar(30)	Use the code value associated with the flag. For example, the 'AL', flag in this example had a description value of 'Alumni'.
@user_id	varchar(20)	We recommend that you use a name here, such as the name of your API application, for auditing purposes. The user_id value does not need to match the name of an actual DP user account.

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?action=dp_saveflag_xml&params=135,'AL','xmluser'&apikey=xxxxxx
```

Returns:

```
<result>
-<record>
<field name="" id="" value=" 135" />
</record>
</result>
```

Notes:

- 135 is the donor_id associated with the updated value
- This example assumes that a user defined field called ALT_PHONE already exists in your system
- If the flag was already set, the system returns an error like this:
[Microsoft][ODBC SQL Server Driver][SQL Server]Violation of PRIMARY KEY constraint 'PK_dpflags'. Cannot insert duplicate key in object
- To view the flags set for a specified donor, use this command format:

https://www.donorperfect.net/prod/xmlrequest.asp?action=select * from dpflags where donor_id=135&apikey=xxxxxxxxxxxxxxxxxx

dp_delflags_xml

Introduction

This procedure **removes (deletes) all flags for the specified donor**. Flags are shown on the main donor screen in DP.

Parameters:

Parameter	Type	Notes
@donor_id	numeric	Specify the donor_id of the donor for whom the flags (all of them) are to be deleted
@user_id	varchar(20)	

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?action=dp_delflags_xml&parameters=135,'xmluser'&apikey=xxxxxx
```

Returns:

```
<result>
<record>
<field name="" id="" value="135"/>
</record>
</result>
```

Notes:

- In this example, 135 is the donor_id associated with the deleted flags
- It is not currently possible to delete individual flags for a specified donor. This command deletes all flags set for the specified donor_id.
- To view the flags set for a specified donor, use a SELECT query to retrieve a list of all set flags for the specified donor. You will use this list of set flags to re-set all flags you did not want un-set for the specified donor: SELECT * FROM DPFLAGS WHERE DONOR_ID={desired donor_id}
- See dp_saveflag_xml for information on setting flags.

dp_tribAnon_MyTribSummary

Introduction

This API call has been added to support the new DP tribute functionality. It retrieves either a list of

- All tributes **or**
- All Active tributes (<field id="ActiveFlg" value="True" name="ActiveFlg")

Parameters

Parameter	Type	Notes
@ShowAllRecords	numeric	If set to 1, retrieves all tributes If set to 0, retrieves all ACTIVE tributes but does not include inactive tributes - i.e.; tributes where the ActiveFlg='False'
@userId	numeric	Enter null. This parameter is not applicable to API calls.

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxxxxxxxx&action=dp_tribAnon_MyTribSummary&params=@showallrecords=1,@userId=null
```

Returns:

```
<record>  
<field name="TributeID" id="TributeID" value="4"/>  
<field name="ActiveFlg" id="ActiveFlg" value="True"/>  
<field name="Type" id="Type" value="In Honor Of"/>  
<field name="Name" id="Name" value="In Honor of Ishmael"/>  
<field name="Created" id="Created" value="8/22/2011"/>  
<field name="Gift_Amt" id="Gift_Amt" value="534"/>  
<field name="Gift_Count" id="Gift_Count" value="12"/>  
<field name="Notif_Name" id="Notif_Name" value="Mark Panagaris"/>  
</record>  
...
```

Notes

The User Identification Number parameter must be entered as 'null' to process this API call.

Either of these two parameter combinations work (depending on whether you want all tributes or just the active ones:

- ¶ms=0,null
- ¶ms=1,null

dp_tribAnon_Search

Introduction

This API call allows you to search for a tribute by the tribute name and either include or exclude inactive tributes.

Parameters

Parameter	Type	Notes
@Keywords	varchar(200)	Enter all or part of the tribute name to retrieve data on each matching tribute.
@IncludeInactive	Int()	Enter 1 to include inactive tributes (ActiveFlg=False) or enter 0 to exclude these.

Sample Call

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxx&action=dp_tribAnon_Search&params=@keywords='Trib',@includeinactive=1
```

Returns

```
<record>
<field name="TributeID" id="TributeID" value="8"/>
<field name="Name" id="Name" value="My New Tribute"/>
<field name="DPCode_ID" id="DPCode_ID" value="707"/>
<field name="ActiveFlg" id="ActiveFlg" value="True"/>
<field name="UserCreatedT" id="UserCreatedT" value="4/1/2017"/>
<field name="Created_Date" id="Created_Date" value="4/1/2017 11:23:00 AM"/>
<field name="Legacy_ID" id="Legacy_ID" value=""/>
<field name="CodeDescription" id="CodeDescription" value="In Honor Of"/>
</record>
<record>
<field name="TributeID" id="TributeID" value="9"/>
<field name="Name" id="Name" value="My New Tribute"/>
<field name="DPCode_ID" id="DPCode_ID" value="707"/>
<field name="ActiveFlg" id="ActiveFlg" value="False"/>
<field name="UserCreatedT" id="UserCreatedT" value="4/1/2017"/>
```

```
<field name="Created_Date" id="Created_Date" value="4/4/2017 3:26:29 PM"/>
<field name="Legacy_ID" id="Legacy_ID" value=""/>
<field name="CodeDescription" id="CodeDescription" value="In Honor Of"/>
</record>
```

Notes

If you already have a tribute that has the same type (CodeDescription) as one you are trying to create, you can use the matching DPCode_ID value.

Example from above: If you needed to create a new tribute that is an 'In Honor Of' tribute, you can safely assume the code of 707 is the correct DPCodeID value in the dp_tribAnon_Create API call for a new tribute of the same type.

dp_tribAnon_Create

Introduction

This API call is used to create a new tribute in DonorPerfect Online (DP) .

Parameters

Parameter	Type	Notes
@Name	varchar(200)	Specify the name that will be used for the tribute
@DPCodeID	int()	This is the numeric code_ID value that is associated with the tribute type. The standard values are M (In Memory Of) and H (In Honor Of) but you will not be specifying the letter value here but rather the <u>numeric</u> value of the code_ID. You can get the required @code_id value with this SQL SELECT query: <pre>SELECT CODE, CODE_ID, DESCRIPTION FROM DPCODES WHERE FIELD_NAME = 'MEMORY_HONOR'</pre> Run the query and record the CODE_ID values. They will not change for your DonorPerfect system. If you are connecting to multiple DonorPerfect systems, you will need to run this once for each system you are connecting to and store the values.
@ActiveFlg	Bit()	Enter 1 here to make the tribute active or 0 to make it inactive.

@UserCreateDt	Datetime()	Enter the current date in this format: mm/dd/yyyy and place single quotes around the date. Entry of time values is NOT supported.
@Recipients	Numeric()	Enter either the donor_id of a single recipient OR multiple donor_id values separated by the pipe symbol and wrapped in single quotes. See examples below. The second example shows four donor ID numbers (11101, 22202, 33303, 44404) being assigned as Recipients and then in the Returns section below that, you can see the names of the donors who correspond to those donor ID numbers. Note: It is possible to create a tribute without specifying any @recipients by omitting this parameter. Any recipients specified here will be notified of all gifts to the tribute from the time that their recipient donor_id is added. (i.e.; if a new recipient is added later on, it won't notify them of gifts received before they are added)

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxxxxxxx
&action=dp_tribAnon_Create&params=@name='My First Tribute',
@dpcodeid=708, @activeflg=1, @usercreatedt='03/15/2017', @recipients=575

https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxxxxxxx&actio
n=dp_tribAnon_Create&params=@name='My Second Tribute', @dpcodeid=707,
@activeflg=1, @usercreatedt='03/15/2017',
@recipients='11101|22202|33303|44404'
```

Returns:

```
<result/>
```

Notes

Sample returned @dpcodeid values from a SQL SELECT query:

```
SELECT CODE, CODE_ID, DESCRIPTION FROM DPCODES WHERE FIELD_NAME =
'MEMORY_HONOR'.
```

You need to run this query for your DP system to determine the @DPCodeID values you need for your DP system :

```
<result>
<record>
<field name="CODE" id="CODE" value="H"/>
<field name="CODE_ID" id="CODE_ID" value="707"/>
<field name="DESCRIPTION" id="DESCRIPTION" value="In Honor Of"/>
</record>
```

```

<record>
<field name="CODE" id="CODE" value="M"/>
<field name="CODE_ID" id="CODE_ID" value="708"/>
<field name="DESCRIPTION" id="DESCRIPTION" value="In Memory Of"/>
</record>
</result>

```

dp_tribAnon_AssocTribToGift

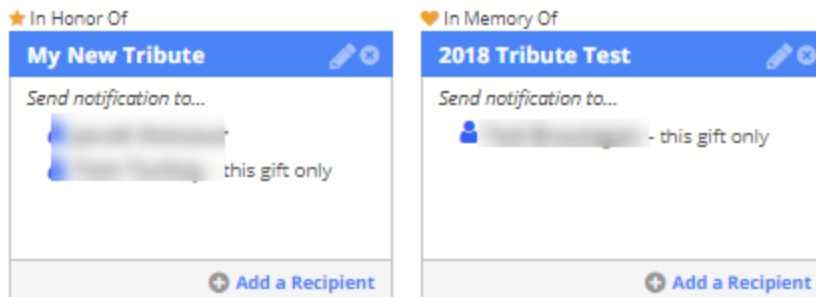
Introduction

This API call is used to create the association between a gift and a tribute by specifying the GIFT_ID and the TributeID_List

Once you have associated a tribute to a gift, the tribute will show in the Tribute Details section of the gift screen.

Tribute Details

Tribute(s) Applied to this Gift...



Parameters

Parameter	Type	Notes
@Gift_ID	numeric	Specify the GIFT_ID of the gift that will be associated with the specified tribute
@TributeID_List	numeric	Enter the TributeID of the tribute. You can optionally enter a comma separated list of tribute ID numbers - e.g.; 9, 12, 33 Tribute IDs are included in the data retrieved in the dp_tribAnon_MyTribSummary API call.

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxx&action=dp_tribAnon_AssocTribToGift&params=@giftid=10561, @tributeid_list=12
```

Returns:

```
<result>  
<record>  
<field name="TributeID" id="TributeID" value="12"/>  
</record>  
</result>
```

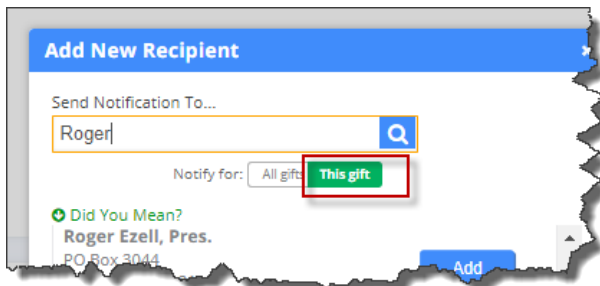
Notes

Use the dp_tribAnon_MyTribSummary API call to retrieve the TributeID values.

dp_tribAnon_SaveTribRecipient

Introduction

Use this API call to add a new recipient notification for a particular gift (This gift):



When you have a gift that already has tribute(s) applied to it, the user interface shows an option to **Add a Recipient**. The resulting window gives you the opportunity to send a notification to an additional recipient, and specifically, for just this gift. This API call supports that functionality.

Tribute(s) Applied to this Gift...

★ In Honor Of

My New Tribute

Send notification to...

[Name - [Redacted]]
 [Name - [Redacted]]

+ Add a Recipient

Add New Recipient

Send Notification To...

Roger

Notify for: ☐ All gifts ☒ **This gift**

Did You Mean?
 Roger Ezell, Pres.
 PO Box 3044

Add

Parameters

Parameter	Type	Notes
@DonorId	numeric	The Donor ID number of the new tribute notification recipient. Example, if you are notifying Susan about Frank's gift, you would specify Susan's Donor_ID number here
@TributeID	numeric	The ID number of the tribute
@GiftID	numeric	The Gift ID number of the gift this donor will be notified about. Per example above, you would specify the Gift_ID number of Fred's gift here.
@Level	varchar (20)	Always set this to 'L'

Sample Call

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxxxxxxx
&action=dp_tribAnon_SaveTribRecipient&params=@donorid=640, @tributeid=9,
@giftid=12879, @level='L'
```

Notes

To use this API call, it is **important** to use the named field syntax shown in the example – e.g; @this=1, @that=2,.

To locate the recipient in the DP user interface

- Go to Tributes under the cog icon (top right hand corner of user interface)

- Search for and select the tribute by its Tribute ID number (above)
- You should see the recipient donor under the 'Send Notifications To' text

The `dp_tribNotif_Save` API call should be run after the `dp_tribAnon_SaveTribRecipient` API call to make sure that the actual notification record is created for the recipient of the notification.

dp_tribNotif_Save

Introduction

This API call creates a tribute gift notification record for a person who is to be notified of a tribute. The tribute gift notification records added with this API call are for the notification recipient for **this gift only**.

Parameters

Parameter	Type	Notes
@Gift_ID	numeric	Enter 0 to create a new tribute gift notification record or the gift_id number of an existing tribute gift notification record if you are updating one.
@Donor_Id	numeric	The Donor ID number of the new notification record recipient. Please note that this may be a different donor than the person who gave the original tribute gift.
@glink	numeric	The Gift ID number of the gift this donor will be notified about
@tlink	numeric	The Tribute ID number of the tribute
@smount	Money	Enter the amount of the gift
@total	Money	Enter the amount of the gift
@bill	Money	Enter 0.00
@start_date	Date	Enter null
@frequency	Varchar	Enter null
@gift_type	Varchar	Enter 'SN'
@record_type	Varchar	Enter 'N' to indicate this is a notification record

@gl_code	Varchar	Enter same value that was set for the original gift
@solicit_code	Varchar	Enter same value that was set for the original gift
@sub_solicit_code	Varchar	Enter same value that was set for the original gift
@campaign	Varchar	Enter same value that was set for the original gift
@ty_letter_no	Varchar(30)	Enter 'NT' for this field.
@fmv	Money	Enter same value that was set for the original gift
@reference	Varchar	Enter SafeSave transaction record or null
@gfname	Varchar(50)	null
@glname	Varchar(75)	null
@gift_narrative	Nvarchar(4000)	If desired, enter gift narrative text
@membership_type	Varchar	Null
@membership_level	Varchar	Null
@membership_enr_date	Date	Null
@membership_exp_date	Date	Null
@address_id	Numeric	Enter 0 unless the notification letter is to go to an address other than the one shown on the Main screen.
@user_id	Varchar(20)	Enter the name of your API application - max 20 characters

Sample Call

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxx&action=dp_tribNotif_Save&params=@Gift_ID=0, @Donor_Id=642, @glink=12879, @tlink=12, @amount=23.23, @total=23.23, @bill=0.00, @start_date=null, @frequency=null, @gift_type='SN', @record_type='N', @gl_code='MIS', @solicit_code='BD04', @sub_solicit_code=null, @campaign=null, @ty_letter_no='NT', @fmv=0.00, @reference=null, @gfname=null, @glname=null, @gift_narrative='This is a gift narrative comment', @address_id=0, @user_id='APIApp'
```

Returns

```
<result>
<record>
<field name="" id="" value="12911"/>
</record>
</result>
```

Notes

The notification shows up as a new gift for donor ID 642 with the Type of Gift field set to Send Notification.

The dp_tribNotif_Save API call should be run after the dp_tribAnon_SaveTribRecipient API call to make sure that the actual notification record is created for the recipient of the notification.

Tribute gift notification records will show in the Linked Gift screen of the original gift (i.e.; the gift_id specified in the @glink field in a table called Linked Gifts. This table will include all notification gifts for all tributes linked to this gift.

dp_tribAnon_Update

Introduction

User this API call if you need to update the list of recipients for all gifts that are associated with a particular tribute. This API call is different than the dp_tribAnon_SaveTribRecipient and dp_tribNotif_Save API calls which add a recipient to a tribute for one specified gift.

It is API equivalent to the Add Recipient link in the Edit Tribute screen (shown below):

Provide Tribute Details

Type*

In Honor Of 

Name*

In Honour of Ishmael

Date*

04/08/2019 ☒ Active**Send Notifications To...** Mark Panagaris   Tom Turkey   Ted Brautigan   Add Recipient**Parameters**

Parameter	Type	Notes
@TributeID	Numeric ()	Enter the ID number of the tribute you are updating
@name	Nvarchar (200)	Enter the existing name of the tribute to be updated. See Notes section below for example of a SELECT statement that will give you this info.
@dpcode_id	Numeric ()	This is the numeric code_ID value that is associated with the tribute type. See Notes section below for example of a SELECT statement that will give you this info.
@ActiveFlg	Bit	Set as 1 for True (active) or 0 for False (inactive)
@UserCreatedDt	Date	Enter existing user create date in this format: 'MM/DD/YYYY'
@recipients	Numeric()	Enter the list of all existing and any new recipients using the pipe symbol as a delimiter. ALSO, prefix the list with capital letter N and also wrap the recipients list in single quotes. Example: @recipients=N'105 43256 323387 137'

Sample Call

```
... &action=dp_tribAnon_Update&params=@tributeID=4,@name='In Honour of  
Ishmael',@DPCode_id=707,@ActiveFlg=1,@UserCreateDt='04/08/2019',@Recipien  
ts=N'135|640|642'
```

Returns

The donor_id numbers you supplied in the API call are replaced with the names associated with the specified donor_id numbers:

```
<result>  
<record>  
<field name="TributeID" id="TributeID" value="4"/>  
<field name="Name" id="Name" value="In Honour of Ishmael"/>  
<field name="ActiveFlg" id="ActiveFlg" value="True"/>  
<field name="UserCreateDt" id="UserCreateDt" value="4/8/2019"/>  
<field name="DPCode_ID" id="DPCode_ID" value="707"/>  
<field name="CodeDescription" id="CodeDescription" value="In Honor Of"/>  
<field name="NotificantNames" id="NotificantNames" value="Ted Brautigan,  
Mark Panagaris, Tom Turkey"/>  
</record>  
</result>
```

Notes

IMPORTANT: You need to first retrieve info on the existing tribute @recipients as you must include these in the dp_tribAnon_Update call otherwise they will no longer be associated with the tribute.

Here is an example of a SELECT statement that will give you the field values you will need for an existing tribute:

```
...&action=select r.tributeid, t.name, r.donor_id, r.createddate,  
t.activeflg from dptributesanon t, dptributesanon_recipients r where  
t.tributeid=r.tributeid and t.tributeid={desired tributeid number}
```

It is not necessary to follow this API call with the dp_tribNotif_Save API call.

dp_PaymentMethod_Insert

Introduction

This procedure allows insertion of DP Payment Method values. This table is used on systems with the EFT Transactions feature enabled.

This procedure will save a single parameter for a specified User Defined Field (UDF).

Parameters:

Parameter	Type	Notes
@CustomerVaultID	Nvarchar(55)	Enter -0 to create a new Customer Vault ID record
@donor_id	Int()	
@IsDefault	Bit	Enter 1 if this is will be the default EFT payment method
@AccountType	Nvarchar(256)	e.g. 'Visa'
@dpPaymentMethodTypeID	Nvarchar(20)	e.g.; 'creditcard'
@CardNumberLastFour	Nvarchar(16)	e.g.; '4xxxxxxxxxxx1111'
@CardExpirationDate	Nvarchar(10)	Enter retrieved value or other value as appropriate Format: 'MMYY' e.g.; '0923' for Sept. 2023
@BankAccountNumberLastFour	Nvarchar(50)	
@NameOnAccount	Nvarchar(256)	
@CreatedDate	datetime	Set as 'MM/DD/YYYY' only. Setting time values is <u>not</u> supported.
@ModifiedDate	datetime	Set as 'MM/DD/YYYY' only. Setting time values is <u>not</u> supported.
@import_id	int	
@created_by	Nvarchar(20)	
@modified_by	Nvarchar(20)	
@selected_currency	Nvarchar(3)	e.g 'USD', 'CAD', per default currency used by the DonorPerfect client

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?action=dp_paymentmethod_insert&params=0,147,1,'Visa','creditcard','4xxxxxxxxxx1234','0412',null,'Oriana',null,null,null,null,null,'USD'&apikey=xxxxxx
```

Returns:

```
<result>
-<record>
<field name="DpPaymentMethodID" id="DpPaymentMethodID" value="2"/>
</record>
</result>
```

Notes:

- This table would normally only be populated from an ecommerce API where the DP system has EFT Transactions enabled.

Here are two samples of XML data retrieved from a Dynamic SELECT Query on the DpPaymentMethod table from a system with EFT Transactions enabled. :

Sample Check Item:

```
- <record>
  <field name="DpPaymentMethodID" id="DpPaymentMethodID" value="402" />
  <field name="donor_id" id="donor_id" value="123456" />
  <field name="IsDefault" id="IsDefault" value="True" />
  <field name="AccountType" id="AccountType" value="Bank Account" />
  <field name="dpPaymentMethodTypeID" id="dpPaymentMethodTypeID"
value="check" />
  <field name="CardNumberLastFour" id="CardNumberLastFour" value="" />
  <field name="CardExpirationDate" id="CardExpirationDate" value="" />
  <field name="BankAccountNumberLastFour" id="BankAccountNumberLastFour"
value="7xxxx6543" />
  <field name="NameOnAccount" id="NameOnAccount" value="Arthur
Roundtable" />
  <field name="CreatedDate" id="CreatedDate" value="10/15/2010 12:01:48
PM" />
  <field name="ModifiedDate" id="ModifiedDate" value="10/15/2010 12:01:46
PM" />
  <field name="CustomerVaultID" id="CustomerVaultID" value="1742923032"
/>
  <field name="import_id" id="import_id" value="" />
  <field name="created_by" id="created_by" value="" />
  <field name="modified_by" id="modified_by" value="" />
  <field name="selected_currency" id="selected_currency" value="CAD" />
</record>
```

Sample Credit Card Item:

```
- <record>
  <field name="DpPaymentMethodID" id="DpPaymentMethodID" value="392" />
  <field name="donor_id" id="donor_id" value="21245" />
  <field name="IsDefault" id="IsDefault" value="True" />
  <field name="AccountType" id="AccountType" value="MasterCard" />
  <field name="dpPaymentMethodTypeID" id="dpPaymentMethodTypeID"
value="creditcard" />
  <field name="CardNumberLastFour" id="CardNumberLastFour"
value="1xxxxxxxxxxx2345" />
  <field name="CardExpirationDate" id="CardExpirationDate" value="0614"
/>
  <field name="BankAccountNumberLastFour" id="BankAccountNumberLastFour"
value="" />
  <field name="NameOnAccount" id="NameOnAccount" value="" />
  <field name="CreatedDate" id="CreatedDate" value="10/14/2010 1:11:30
PM" />
  <field name="ModifiedDate" id="ModifiedDate" value="10/14/2010 1:11:29
PM" />
```

```
<field name="CustomerVaultID" id="CustomerVaultID" value="1234567890"
/>
<field name="import_id" id="import_id" value="" />
<field name="created_by" id="created_by" value="Diane Warner" />
<field name="modified_by" id="modified_by" value="" />
<field name="selected_currency" id="selected_currency" value="USD" />
</record>
```

dp_PaymentMethod_Update

Introduction

This procedure is used to update value(s) for an existing Payment Method (see Accounts tab in user interface).

To use this procedure

You will need to first retrieve all existing values for the payment method. Here is an example of a SELECT query that pulls all values for a specified CustomerVaultID:

```
&action=select * from dppaymentmethod where customervaultid=1205299919
```

- You can then use the retrieved values along with any updated values in the dp_PaymentMethod_Update API call

Parameters:

Parameter	Type	Notes
@CustomerVaultID	nvarchar(55)	Use retrieved @CustomerVaultID value
@donor_id	int	Use retrieved @donor_id value
@IsDefault	int	Enter 1 (True) if this payment method is to be the default or 0 (False) if it will not be the default
@AccountType	nvarchar(256)	Enter retrieved value or other value as appropriate
@dpPaymentMethodTypeID	nvarchar(20)	Enter retrieved value or other value as appropriate
@CardNumberLastFour	nvarchar(16)	Enter retrieved value or other value as appropriate
@CardExpirationDate	nvarchar(16)	Enter retrieved value or other value as appropriate Format: 'MMYY'
@BankAccountNumberLastFour	nvarchar(50)	Enter retrieved value or other value as appropriate
@NameOnAccount	nvarchar(256)	Enter retrieved value or other value as appropriate
@ModifiedDate	datetime	Enter today's date in format 'MM/DD/YYYY'

Sample Call:

```
https://www.donorperfect.net/prod/xmlrequest.asp?apikey=xxxxxx&action=dp
_PaymentMethod_Update&params=@customervaultid=1205299919, @donor_id=135,
@IsDefault=1, @AccountType='Visa', @dpPaymentMethodTypeID='creditcard',
@CardNumberLastFour='4567', @CardExpirationDate='0923',
@BankAccountNumberLastFour=null, @NameOnAccount=null,
@ModifiedDate='05/18/2023', @modified_by='API Guy'
```

Returns:

```
<record>

<field name="DpPaymentMethodID" id="DpPaymentMethodID" value="224"/>

<field name="donor_id" id="donor_id" value="135"/>

<field name="IsDefault" id="IsDefault" value="True"/>
```

```
<field name="AccountType" id="AccountType" value="Visa"/>

<field name="dpPaymentMethodTypeID" id="dpPaymentMethodTypeID"
value="creditcard"/>

<field name="CardNumberLastFour" id="CardNumberLastFour" value="4567"/>

<field name="CardExpirationDate" id="CardExpirationDate" value="0923"/>

<field name="BankAccountNumberLastFour" id="BankAccountNumberLastFour"
value=""/>

<field name="NameOnAccount" id="NameOnAccount" value=""/>

<field name="CreatedDate" id="CreatedDate" value="4/9/2019"/>

<field name="ModifiedDate" id="ModifiedDate" value="5/18/2023 10:31:23
AM"/>

<field name="CustomerVaultID" id="CustomerVaultID" value="1205299919"/>

<field name="import_id" id="import_id" value=""/>

<field name="created_by" id="created_by" value=""/>

<field name="modified_by" id="modified_by" value="API Guy"/>

<field name="selected_currency" id="selected_currency" value=""/>

<field name="wl_import_id" id="wl_import_id" value="0"/>

<field name="notFoundonGateway" id="notFoundonGateway" value=""/>

<field name="last_sync_date" id="last_sync_date" value=""/>

</record>
```

Notes:

You **MUST** reload all existing (or updated existing) values back into DonorPerfect even if you are not changing the existing value

5. Supplemental Information

Gift Aid Program

The Gift Aid program is a tax incentive program used in the United Kingdom to encourage people to give to registered charities. DonorPerfect Online provides these Gift Aid fields.

Table	Field	Notes
DP	GIFT_AID_ELIGIBLE	'Y' or 'NO' to indicate whether this donor is gift aid eligible
DPUDF	GA_TITLE	Per title field in gift aid charity donation
DPUDF	GA_ADDRESS	Text box for foreign (non-UK) address
DPGIFT	GIFT_AID_ELIGIBLE_G	'Y' or 'N' to indicate whether the gift is gift aid eligible
DPGIFT	GIFT_AID_DATE	Date when gift was submitted to HM Revenue & Customs
DPGIFT	GIFT_AID_AMT	Portion of gift eligible for gift aid tax relief
DPTHERINFO	GA_START_DT	Start date of gift aid declaration
DPTHERINFO	GA_END_DT	End date of gift aid declaration
DPTHERINFO	GA_DECL_ACTIVE	'Y' or 'N' to indicate whether the gift aid declaration is active

If you are not sure what the United Kingdom Gift Aid program is or how it works, please consult the United Kingdom Government site for more information.

Soft Credits

To create a soft credit gift, create that gift with RECORD_TYPE='S' and use the GLINK field to connect this to the standard gift (record_type='G') by its GIFT_ID number .

Here's an example: Donor_id 1336 actually gave \$10, but we recognize that they gave because of donor_id 640 so we create the soft credit back to donor_id 640. Here's a SELECT showing the relevant fields:

```
<result>
<record>
<field name="donor_id" id="donor_id" value="1336"/>
<field name="gift_id" id="gift_id" value="12857"/>
```

```
<field name="amount" id="amount" value="10"/>
<field name="record_type" id="record_type" value="G"/>
<field name="glink" id="glink" value=""/>
</record>

<record>
<field name="donor_id" id="donor_id" value="640"/>
<field name="gift_id" id="gift_id" value="12858"/>
<field name="amount" id="amount" value="10"/>
<field name="record_type" id="record_type" value="S"/>
<field name="glink" id="glink" value="12857"/>
</record>
</result>
```

Split Gifts

A split gift in DonorPerfect is used when you need to identify the destination (e.g.; the GL Code) of parts of the gift for different purposes. Example: a \$100 gift where \$75 is going to the building fund and the other \$25 is going to the general fund.

If you need to create split gifts in DonorPerfect

- The main gift (\$100) is identified with record_type='M'
- Each of the splits (\$75 / \$25) are identified with record_type='G'
- Each of the splits have a glink field value which is the gift_id number of the main gift
- Each of the splits have the SPLIT_GIFT='Y' but the main gift has this field set to 'N'
- For any financial fields that are to be identified in the splits, set the equivalent **main** gift to 'SEE_SPLIT' and identify the appropriate code values in each of the splits. Example of main gift:
 - @gl_code='SEE_SPLIT', @solicit_code='SEE_SPLIT',
@sub_solicit_code='SEE_SPLIT', @campaign='SEE_SPLIT',

Example:

```

▼<result>
  ▼<record>
    <field name="donor_id" id="donor_id" value="640"/>
    <field name="gift_id" id="gift_id" value="12933"/>
    <field name="record_type" id="record_type" value="M"/>
    <field name="amount" id="amount" value="100"/>
    <field name="gl_code" id="gl_code" value="SEE_SPLIT"/>
    <field name="split_gift" id="split_gift" value="N"/>
    <field name="glink" id="glink" value=""/>
  </record>
  ▼<record>
    <field name="donor_id" id="donor_id" value="640"/>
    <field name="gift_id" id="gift_id" value="12937"/>
    <field name="record_type" id="record_type" value="G"/>
    <field name="amount" id="amount" value="40"/>
    <field name="gl_code" id="gl_code" value="4540-N"/>
    <field name="split_gift" id="split_gift" value="Y"/>
    <field name="glink" id="glink" value="12933"/>
  </record>
  ▼<record>
    <field name="donor_id" id="donor_id" value="640"/>
    <field name="gift_id" id="gift_id" value="12938"/>
    <field name="record_type" id="record_type" value="G"/>
    <field name="amount" id="amount" value="60"/>
    <field name="gl_code" id="gl_code" value="EDU-R"/>
    <field name="split_gift" id="split_gift" value="Y"/>
    <field name="glink" id="glink" value="12933"/>
  </record>
</result>

```

Pledge Notes

When you create a pledge in DonorPerfect Online to represent a recurring gift, you create a parent gift that represents the overall commitment – e.g; Dave Smith pledges to give \$40 every month. If Dave also has opted to give his first \$40 gift when he set up the pledge, this \$40 gift is a second gift separate from the parent pledge – but - associated with the parent pledge.

So in this scenario,

- a \$40 per month pledge gift is created with a RECORD_TYPE field value of 'P' for pledge.
- The initial \$40 gift Dave gives when he's making the pledge comes in as a second gift with RECORD_TYPE = 'G' for gift. Also, the PLINK field of this gift gets the GIFT_ID value of the parent pledge so that it this gift is linked to this specific pledge
- Also, to allow DonorPerfect to use its' EFT transactions functionality so that subsequent pledge payments can be run from within the DP user interface, the parent pledge gift must have the DPGIFTUDF.EFT field set to 'Y'. This is done using the **dp_save_udf_xml** API call.
- Also, make sure you populate the @vault_id field with the Vault ID number returned by SafeSave.

- Note: The EFT field is shown as the Monthly Giving field in the user interface on the Pledge screen:
 - `dp_save_udf_xml¶ms={parent_pledge_gift_id},EFT,'C',Y,null,null,'API User'`
- DonorPerfect will also need an associated Payment Method to process pledge payments. Payment Method(s) are displayed under the Accounts tab in the DonorPerfect screen interface.
 - Payment methods are set up using the `dp_PaymentMethod_insert` API call
 - To use this API call, you will first need to create a Customer Vault ID in your SafeSave account using the SafeSave API
 - Creating the payment method with it's associated customer vault ID information in DonorPerfect allows all subsequent pledge payments to be managed through the DonorPerfect user interface without the need for any subsequent API application coding. It's like telling DonorPerfect "Here's a pledge, here's how much it's for, how often to collect payments and a payment method you can use for the payments."

Checkbox Fields

These are stored in a table called **DPUSERMULTIVALUES** where the **MATCHING_ID** field is equal to the **DONOR_ID** field.

Typical Example: The Donor Interests checkbox fields in the BIO tab.

Instances in this table are only created for items that are checked, so if, for example, a donor isn't interested in Disaster Relief (coded as 'DR') and the box hasn't been checked, then there will be no entry in this table for **DONINTS** for 'DR' for this **matching_id** (**donor_id**).

Example:

Retrieve all checked items for donor 12345 for an Events checkbox:

- `&action=select field_name, matching_id, code from dpusermultivalues where field_name='EVENTS' and matching_id=12345`

Caution: Flags are not the same as checkboxes: Find flags in DPFLAGS – e.g; `select * from dpflags where donor_id=xxx`

Sample output:

```
<record><field name="field_name" id="field_name" value="EVENTS"/><field name="matching_id" id="matching_id" value="12345"/><field name="code" id="code" value="AB2140"/></record>
```

Get a list of possible field values

To determine the appropriate CODE value and FIELD NAME, use the Code Maintenance screen in DonorPerfect Online and select the desired field (e.g.; 'Donor Interests / Bio Screen').

When/if you want to retrieve a list of all possible values (i.e.; not just the ones that are set), you can get these from the DPCODES table.

Example:

Retrieving possible CODE values (and also the Description values in case you need to display the selection label on your site somewhere) for a checkbox field with FIELD_NAME='XXX'&action=select code, description from dpcodes where field_name='XXX'

If you aren't sure what the field_name is, go into the Code Maintenance screen on your client system and select the field you want in the 'Show only' dropdown.

Set a value

Use the **mergemultivalues** or **dp_savemultivalue_xml** API call per API User Manual

Delete a value

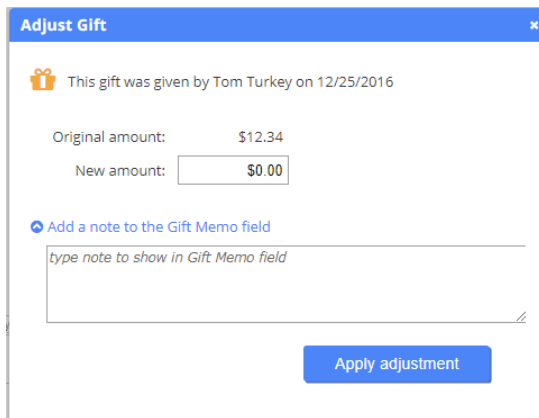
Use the **dp_deletemultivalues_xml** API call

Gift Adjustments

You can apply gift adjustments using the API if your application is handling refunds.

In the user interface, Gift Adjustments are done using the Adjust Gift screen:





Adjust Gift x

 This gift was given by Tom Turkey on 12/25/2016

Original amount: \$12.34

New amount:

☒ Add a note to the Gift Memo field

type note to show in Gift Memo field

If you are using the API to do this, you will

- make adjustments to the **Original Gift** (gift_id 12885 in this example)
- create a new **Adjustment Gift** (gift_id 12891 in this example)
- create a new **Resulting Gift** (gift_id 12890 in this example)

Your gift_id numbers will, of course, be different than the ones shown here:

Original Gift:

- In this example, the original gift_id number is 12885
- Use the dp_savegift API call to set the @record type value of A
 - This will automatically update the @modified_date to today's date and the @modified_by field to the @user_id field value you specify.
 - When you are using dp_savegift API call you will want to first use a SELECT query to retrieve all the values in this gift that you are 'updating' to the existing values
 - When dp_savegift is used to update a gift rather than to create a new one, use the existing gift gift_id value in the @gift_id field instead of zero
- This gift shows original amount (29.95)

Adjustment Gift

- Use dp_savegift to create this new Adjustment Gift
- The new Adjustment gift gift_id number in this example is 12891
- Set today's date as the gift_date
- Record type in this case of H
- Set the amount is just the adjustment amount (-10)
- Include all the field data from the original (e.g.; GL Code, Solicit Code, Thank-You Letter, etc.)
- Set the @glink field to the gift_id number of the Original Gift

Resulting Gift

- Use dp_savegift to create this new Resulting Gift
- The new Resulting gift gift_id number in this example is 12890
- This gift gets the Gift Memo (gift_narrative) note appended to the end of any existing gift_narrative field contents from the original gift so you will need to retrieve the Original Gift gift_narrative field contents.
- Include all the field data from the original (e.g.; GL Code, Solicit Code, Thank-You Letter, etc.)
- Set the amount of this gift as the New Amount (19.95)
- Set the record_type to G unless the adjustment took the new amount to \$0.00 in which case you set this gift to have record_type of C (or J if the original gift is a split gift)
- Set the @alink field to the gift_id number of the Original gift
- **Please Note:** If the original gift included additional field data (e.g.; financial field data or User Defined Field data), the Resulting gift should be set with the same values

Example retrieved results:

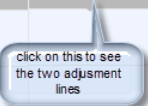
```
<result>
<record>
This is the ORIGINAL gift:
<record>
<field name="donor_id" id="donor_id" value="640"/>
<field name="gift_id" id="gift_id" value="12885"/>
<field name="record_type" id="record_type" value="A"/>
<field name="gift_date" id="gift_date" value="1/10/2018"/>
<field name="amount" id="amount" value="29.95"/>
<field name="created_date" id="created_date" value="1/10/2018"/>
<field name="created_by" id="created_by" value="My App Name"/>
<field name="modified_date" id="modified_date" value="4/5/2018"/>
<field name="modified_by" id="modified_by" value="My App Name"/>
<field name="gift_narrative" id="gift_narrative" value="xxx"/>
<field name="alink" id="alink" value=""/>
<field name="glink" id="glink" value=""/>
</record>
This is the ADJUSTMENT gift:
<record>
```

```
<field name="donor_id" id="donor_id" value="640"/>
<field name="gift_id" id="gift_id" value="12891"/>
<field name="record_type" id="record_type" value="H"/>
<field name="gift_date" id="gift_date" value="4/5/2018"/>
<field name="amount" id="amount" value="-10"/>
<field name="created_date" id="created_date" value="4/5/2018"/>
<field name="created_by" id="created_by" value=" My App Name"/>
<field name="modified_date" id="modified_date" value=""/>
<field name="modified_by" id="modified_by" value=""/>
<field name="gift_narrative" id="gift_narrative" value="xxx"/>
<field name="alink" id="alink" value=""/>
<field name="glink" id="glink" value="12885"/>
</record>
```

This is the RESULTING gift:

```
<record>
<field name="donor_id" id="donor_id" value="640"/>
<field name="gift_id" id="gift_id" value="12890"/>
<field name="record_type" id="record_type" value="G"/>
<field name="gift_date" id="gift_date" value="1/10/2018"/>
<field name="amount" id="amount" value="19.95"/>
<field name="created_date" id="created_date" value="4/5/2018"/>
<field name="created_by" id="created_by" value=" My App Name"/>
<field name="modified_date" id="modified_date" value=""/>
<field name="modified_by" id="modified_by" value=""/>
<field name="gift_narrative" id="gift_narrative" value="xxx This gift was adjusted by
-$10.00 from $29.95 to new amount $19.95"/>
<field name="alink" id="alink" value="12885"/>
<field name="glink" id="glink" value=""/>
</record>
</result>
```

The result of your gift adjustment will look like this in the user interface:



12890	01/10/2018	\$19.95	\$19.95	Building Fund (R): Furnishings	This gift was adjust...
12891	04/05/2018	-\$10.00	-\$10.00	Building Fund (R): Furnishings	
12885	01/10/2018	\$29.95	\$29.95	Building Fund (R): Furnishings	

Created_date and modified_date fields

Although these fields are not accessible directly via the API, these fields are set in the following way:

@created_by / @modified_by

- When you set the @user_id field,
 - this value is placed in the @created_by if it is a new entry (e.g.; a new donor or gift)
 - this value is placed in the @modified_by you are using a predefined procedure (e.g.; dp_savedonor, dp_savegift) to modify an existing entry

@created_date / @modified_date

- If you are creating a new entry, the @created_date is set to {today}
- If you are modifying an existing entry, the @modified_date is set to {today}

Please note: Date fields are set as 'MM/DD/YYYY' only. Setting time values is not supported.

Tributes API Calls

Here's a quick primer on what each of the tribute related API calls does:

API Call	Function
Dp_tribAnon_MyTribSummary	This API call retrieves a list of all existing tributes. There is a switch in the call to exclude inactive tributes if desired.
Dp_tribAnon_Search	Allows you to search for tributes by name
Dp_tribAnon_Create	Use this API call to create a new tribute
Dp_tribAnon_AssocTribstoGift	Associates a specified gift to an individual tribute or to a comma separated list of tributes
dp_tribAnon_SaveTribRecipient	Use this call to specify the donor ID of the person who will be notified of a tribute gift. This API call causes the tribute box for the specified tribute to appear in the tribute section of the gift owners gift screen.

Dp_tribNotif_Save	This API call completes the notification process and adds a notification gift to the person being notified.
-------------------	---

Unicode Support

DonorPerfect systems can be configured to support UNICODE characters on request. DonorPerfect systems that are configured for Unicode text field handling store text data in UCS-2 / UTF-16 character encoding format.

If your API based application needs to send Unicode characters - e.g.; for foreign language text strings that involve special characters (e.g.; Vietnamese, Polish, Chinese, etc.) to DonorPerfect, there are two things you will need to do:

1. Contact your DonorPerfect account representative and request that your DonorPerfect system be set up to support Unicode text field handling.
2. Once the above has been completed, you can access any text fields that may include foreign language characters by inserting the capital letter 'N' after the = sign and before the quoted text string in your text_field_name='text string' API calls.

Example 1 : ... text_field=N'Phúc cho những người nghèo về tinh thần, vì họ là vương quốc thiên đàng.'

Example 2 (using named parameters syntax):

... @text_field=N'Phúc cho những người nghèo về tinh thần, vì họ là vương quốc thiên đàng.'

IMPORTANT NOTES:

1. You CAN use the modified syntax (FIELD_NAME=N'foreign language text here') to send non-Unicode text to DonorPerfect systems that are not set up for Unicode field support.
2. This modified syntax works nicely with the common API Predefined Procedure calls e.g.; dp_savedonor, dp_savegift, dp_savecontact, dp_saveother, etc.
3. The modified =N'xxx' syntax does not currently support the sending of Unicode text in the dp_save_udf_xml API call. The API call still works BUT the text that gets received will just show question marks where any Unicode characters were in the original text.
 - So, 3rd party applications that need to write Unicode text to any User Defined Fields (UDFs), can **not** send Unicode text with the modified =N'xxx' syntax using the dp_save_udf_xml API call
 - There is a temporary work-around for this. Please contact the DonorPerfect API Desk if you need more info on this workaround.

Quebec (QC) Law 25

The province of Quebec, Canada has enacted legislation regarding the recording of personal information belonging to residents of that province (regardless of the location of your organization).

As a result, DonorPerfect has enabled a number of new User Defined Fields (UDFs) for this purpose. These are described here:

<https://softerware.my.site.com/dpcommunity/s/article/List-of-Sections-and-Fields-for-Law-25>

How to access these via the API

This section shows Screen Designer screenshots of the relevant fields along with the associate Table and API Call required to access:

Table: DPUDF

Use dp_save_udf_xml API call to save these with @matching_id=@donor_id

	Main	Privacy and Consent Profile	PM_CONSENT	Consent Given	Yes/No	1	☐	☒	☐	☒	☐	
	Main	Privacy and Consent Profile	PM_CONSENT_DT	Actual Consent Date	Textbox	2		☒	☐	☒	☐	
	Main	Privacy and Consent Profile	PM_CONSENT_EXPIRY	Actual Consent Expiry Date	Textbox	3		☒	☐	☒	☐	
	Main	Privacy and Consent Profile	PM_DELETE	Erasure requested	Yes/No	4	☐	☒	☐	☒	☒	
	Main	Privacy and Consent Profile	PM_DELETE_DT	Erasure requested on	Textbox	5		☒	☐	☒	☐	

DPTHERINFOUDF

Table: DPTHERINFOUDF

Use `dp_save_udf_xml` API call to save these with `@matching_id=@other_id`

	Other	Privacy and Consent Profile	PO_CONSENT	Consent Given	Yes/No	1		☒	☐	☐	☐	
	Other	Privacy and Consent Profile	PO_CONSENT_DT	Consent Date	Textbox	2		☒	☐	☐	☐	
	Other	Privacy and Consent Profile	PO_CONSENT_EXPIRY	Consent Expiry Date	Textbox	3		☒	☐	☐	☐	
	Other	Privacy and Consent Profile	PO_DELETE	Erasure requested	Yes/No	4		☒	☐	☐	☒	
	Other	Privacy and Consent Profile	PO_DELETE_DT	Erasure requested on	Textbox	5		☒	☐	☐	☐	
	Other	Privacy and Consent Profile	PO_EXPORT_DT	Export requested on	Textbox	6		☒	☐	☐	☐	
	Other	Privacy and Consent Profile	PO_NOTES	Notes - Privacy and Consent	Memo	7		☒	☐	☐	☒	

Test for field presence:

Because these fields do not exist on all DonorPerfect systems, you should run this test before your application begins sending any of the `dp_save_udf_xml` API calls:

```
&action=select distinct field_name from sd_field where field_name LIKE
'PM_%' or field_name LIKE 'PO_%'
```

On a system where these fields exist, you should get a response like this:

```
<result>
  <record>
    <field name="field_name" id="field_name" value="PM_CONSENT"/>
  </record>
  <record>
    <field name="field_name" id="field_name" value="PM_CONSENT_DT"/>
  </record>
  <record>
    <field name="field_name" id="field_name" value="PM_CONSENT_EXPIRY"/>
  </record>
  <record>
    <field name="field_name" id="field_name" value="PM_DELETE"/>
  </record>
  <record>
    <field name="field_name" id="field_name" value="PM_DELETE_DT"/>
  </record>
  <record>
    <field name="field_name" id="field_name" value="PM_NOTES"/>
  </record>
  <record>
    <field name="field_name" id="field_name" value="PO_CONSENT"/>
  </record>
  <record>
    <field name="field_name" id="field_name" value="PO_CONSENT_DT"/>
  </record>
  <record>
    <field name="field_name" id="field_name" value="PO_CONSENT_EXPIRY"/>
  </record>
  <record>
    <field name="field_name" id="field_name" value="PO_DELETE"/>
  </record>
  <record>
    <field name="field_name" id="field_name" value="PO_DELETE_DT"/>
  </record>
  <record>
    <field name="field_name" id="field_name" value="PO_EXPORT_DT"/>
  </record>
  <record>
    <field name="field_name" id="field_name" value="PO_NOTES"/>
  </record>
  <record>
    <field name="field_name" id="field_name" value="POLITICALPARTY"/>
  </record>
  <record>
    <field name="field_name" id="field_name" value="POSITION"/>
  </record>
</result>
```

6. Support and Implementation Services

How to contact DP API Support:

Mark Warren – XML API Consultant

Email: api@softerware.com (Fastest path for resolution.)

Phone: 1-888-637-7745

- Normal Working Hours: Monday - Friday, 8:30 a.m. to 5:00 p.m. EST.
- The acknowledgement response time for API support calls is not guaranteed and is independent of any other support guidelines. Calls are answered in the order they are received and under normal circumstances, it's reasonable to expect a response within one business day. For emergencies, send an additional email to support@donorperfect.com.

Your purchase of the API/XML service allows you access to our support staff and we can offer the following type of services at no additional charge:

- Assistance with API Call syntax.
- Recommendations or suggestions to accomplish your task.
- Error message explanations
- Restoring API service due to our own outages.

Provision of the DP API by SofterWare includes:

- The DP API toolkit (commands)
- The DP API documentation
- API support as described above

API Support does not include assistance or consulting services on applications developed by the client organization or an authorized third party integrator. These remain the sole responsibility of the client organization.

As a result, additional chargeable services not covered above include:

- Creating, Reviewing or Testing API calls on behalf of the client.
- Debugging API Code created by the client or the client's authorized third party.
- Creating or editing website code to utilize the DP API.
- Creating or editing any customer specific API documentation

Please contact your Account Manager for any additional questions regarding our support for the DP API.